DCOPF_v5

April 26, 2026

1 Data Aggregation and Preparation

1.0.1 Step 1: Download the IEEE 118-Bus System

methodology citation: Donti, P., Amos, B., & Kolter, J. Z. (2017). Task-based End-to-End Model Learning in Stochastic Optimization. Advances in Neural Information Processing Systems (NeurIPS), 30. (This paper is foundational for bridging neural network load predictions directly with downstream non-linear grid optimization).

[2]:
```
import urllib.request
import pandas as pd
import re

# 1. Download the file
url = "https://raw.githubusercontent.com/power-grid-lib/pglib-opf/master/
 ↪pglib_opf_case118_ieee.m"
filename = "pglib_opf_case118_ieee.m"
print("Downloading IEEE 118-bus case...")
urllib.request.urlretrieve(url, filename)

def parse_matpower_matrix(filepath, matrix_name):
    with open(filepath, 'r') as f:
        content = f.read()

    # Use regex to find the block of text between 'mpc.matrix_name = [' and '];'
    pattern = rf"mpc\.{matrix_name}\s*=\s*\[(.*?)\];"
    match = re.search(pattern, content, re.DOTALL)

    data = []
    # Read line by line, ignoring MATLAB comments (%)
    for line in match.group(1).strip().split('\n'):
        # Strip comments, remove MATLAB semicolons, and clean whitespace
        clean_line = line.split('%')[0].replace(';', '').strip()

        row = [float(x) for x in clean_line.split()]
        data.append(row)

    return pd.DataFrame(data)
```

```python
# ======================================================================
# 3. EXTRACT THE DC-OPF DATA
# ======================================================================

# --- BUS DATA ---
# Col 0: Bus ID | Col 2: Active Demand (Pd) in MW
bus_df = parse_matpower_matrix(filename, 'bus')
nodal_demand = bus_df[[0, 2]].rename(columns={0: 'bus_id', 2: 'Pd_mw'})

# --- GENERATOR DATA (WITH FIXED ALPHAS) ---
# Col 0: Bus ID | Col 1: Base Pg | Col 8: Pmax (MW) | Col 9: Pmin (MW)
gen_df = parse_matpower_matrix(filename, 'gen')
gen_limits = gen_df[[0, 1, 8, 9]].rename(columns={
    0: 'bus_id',
    1: 'Pg_base_mw',
    8: 'Pmax_mw',
    9: 'Pmin_mw'
})

# 1. Calculate available headroom for each generator
gen_limits['headroom_mw'] = gen_limits['Pmax_mw'] - gen_limits['Pg_base_mw']

# 2. Safety clip: Ensure no negative headroom (in case of MATPOWER rounding␣
↪quirks)
gen_limits['headroom_mw'] = gen_limits['headroom_mw'].clip(lower=0.0)

# 3. Calculate the fixed alpha (Participation Factor)
total_headroom = gen_limits['headroom_mw'].sum()

if total_headroom > 0:
    gen_limits['alpha_fixed'] = gen_limits['headroom_mw'] / total_headroom
else:
    # Fallback if no headroom exists system-wide (unlikely in standard cases)
    gen_limits['alpha_fixed'] = 1.0 / len(gen_limits)

print("Fixed Alpha values calculated successfully. Top 5 contributors:")
print(gen_limits[['bus_id', 'Pg_base_mw', 'Pmax_mw', 'alpha_fixed']].
↪sort_values('alpha_fixed', ascending=False).head())

# --- BRANCH DATA ---
# Col 0: From Bus | Col 1: To Bus | Col 3: Reactance (x) | Col 5: Rate A (MW␣
↪limit)
branch_df = parse_matpower_matrix(filename, 'branch')
branch_data = branch_df[[0, 1, 3, 5]].rename(columns={0: 'from_bus', 1:␣
↪'to_bus', 3: 'x_reactance', 5: 'thermal_limit_mw'})
```

```
# Calculate Susceptance (B = 1/x) for your DC-OPF B-matrix
branch_data['B_susceptance'] = 1.0 / branch_data['x_reactance']

# --- GENERATOR COSTS ---
# Col 3: Number of coefficients | Col 5: Linear cost (c1) | Col 6: Constant␣
↪cost (c0)
# Note: PGLib 118 uses linear costs, so n=2 (represented by 2 coefficients)
gencost_df = parse_matpower_matrix(filename, 'gencost')
gen_costs = gencost_df[[5, 6]].rename(columns={5: 'cost_linear', 6:␣
↪'cost_constant'})

print("Extraction Successful! Here is a sample of your branch data:")
print(branch_data.head())
```

```
Downloading IEEE 118-bus case…
Fixed Alpha values calculated successfully. Top 5 contributors:
    bus_id  Pg_base_mw  Pmax_mw  alpha_fixed
29    69.0       591.0   1182.0     0.181427
28    66.0       392.0    784.0     0.120338
44   100.0       326.5    653.0     0.100230
39    89.0       318.5    637.0     0.097774
36    80.0       254.5    509.0     0.078127
Extraction Successful! Here is a sample of your branch data:
   from_bus  to_bus  x_reactance  thermal_limit_mw  B_susceptance
0       1.0     2.0      0.09990             151.0      10.010010
1       1.0     3.0      0.04240             151.0      23.584906
2       4.0     5.0      0.00798             176.0     125.313283
3       3.0     5.0      0.10800             175.0       9.259259
4       5.0     6.0      0.05400             176.0      18.518519
```

1.0.2 2. Download Demand Data

citation: A novel probabilistic gradient boosting model withmulti-approachfeature selection and iterative seasonal trend decomposition for short-term load forecasting Priyesh Saini∗, S.K. Parida

[3]:
```
import pandas as pd
import numpy as np
import requests
import io
import time
from gridstatus import CAISO

START = "2022-02-03"
END   = "2023-02-03"

# --- 1. CAISO 5-minute system demand ---
iso = CAISO()
load = iso.get_load(
```

```
    start=pd.Timestamp(START, tz="US/Pacific"),
    end=pd.Timestamp(END,   tz="US/Pacific"),
)

load = load.rename(columns={"Load": "load_mw", "Time": "timestamp"})
load["timestamp"] = (pd.to_datetime(load["timestamp"])
                       .dt.tz_convert("US/Pacific")
                       .dt.tz_localize(None))
load["timestamp"] = load["timestamp"].dt.round("5min")
load = (load[["timestamp", "load_mw"]]
        .drop_duplicates("timestamp")
        .sort_values("timestamp")
        .set_index("timestamp"))

# ====================================================================
# 2. ROBUST LAX ASOS: Chunked 5-minute temperature + RH
# ====================================================================
def fetch_asos_chunk(start_dt, end_dt, retries=5): # Increased retries just in␣
↪case
    """Fetches a small time chunk from IEM with automatic exponential backoff.
↪"""
    url = (
        "https://mesonet.agron.iastate.edu/cgi-bin/request/asos.py"
        f"?station=LAX&data=tmpf,relh"
        f"&year1={start_dt.year}&month1={start_dt.month}&day1={start_dt.day}"
        f"&year2={end_dt.year}&month2={end_dt.month}&day2={end_dt.day}"
        "&tz=America/Los_Angeles&format=onlycomma&latlon=no&missing=empty"
    )
    headers = {"User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"}

    for attempt in range(retries):
        response = requests.get(url, headers=headers)
        if response.status_code == 200:
            return pd.read_csv(io.StringIO(response.text),␣
↪parse_dates=["valid"])

        # Catch BOTH 503 (Server Busy) and 429 (Too Many Requests)
        elif response.status_code in [503, 429]:
            wait_time = 2 ** (attempt + 1) + 2  # Added a small buffer
            print(f"  [!] HTTP {response.status_code}. Retrying in {wait_time}s.
↪..")
            time.sleep(wait_time)
        else:
            response.raise_for_status()

    raise Exception(f"Failed to fetch {start_dt.date()} after {retries} retries.
↪")
```

```python
print("Fetching weather data in 30-day chunks to prevent IEM server timeouts...")

# Create 30-day intervals to loop through
date_chunks = pd.date_range(start=START, end=END, freq='30D').tolist()
if pd.Timestamp(END) not in date_chunks:
    date_chunks.append(pd.Timestamp(END))

wx_chunks = []
for i in range(len(date_chunks) - 1):
    s = date_chunks[i]
    e = date_chunks[i+1]
    print(f"  -> Pulling chunk: {s.date()} to {e.date()}")

    chunk_df = fetch_asos_chunk(s, e)
    wx_chunks.append(chunk_df)
    time.sleep(3)  # Polite pause between chunks

# Combine all the chunks into one DataFrame
wx = pd.concat(wx_chunks, ignore_index=True)
wx = wx.drop_duplicates(subset=["valid"])

# Continue with your exact original data processing
wx = wx.rename(columns={"valid": "timestamp", "tmpf": "temp_f", "relh": "rh_pct"})
wx["temperature"] = (wx["temp_f"] - 32) * 5/9

T  = wx["temperature"]
RH = wx["rh_pct"]
wx["humidity"] = (6.112 * np.exp(17.67 * T / (T + 243.5)) * RH * 2.1674) / (273.15 + T)

wx = wx[["timestamp", "temperature", "humidity"]].dropna()
wx["timestamp"] = wx["timestamp"].dt.round("5min")
wx = (wx.groupby("timestamp").mean().sort_index())

grid = pd.date_range(load.index.min(), load.index.max(), freq="5min")
wx = wx.reindex(grid).interpolate("time", limit=12)

# --- 3. Merge + sanity check ---
df = load.join(wx, how="inner")
print(f"\nLoad rows:    {len(load):,}")
print(f"Weather rows: {len(wx.dropna()):,}")
print(f"Merged rows:  {len(df):,}   (expect ~105,000 for one year of 5-min)")
```

```
  0%|          | 0/365 [00:00<?, ?it/s]2026-04-26 07:10:30 - INFO - Fetching
URL: https://www.caiso.com/outlook/history/20220203/demand.csv?_=1777201830
  0%|          | 1/365 [00:00<03:08,  1.93it/s]2026-04-26 07:10:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220204/demand.csv?_=1777201830
  1%|          | 2/365 [00:00<02:25,  2.49it/s]2026-04-26 07:10:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220205/demand.csv?_=1777201830
  1%|          | 3/365 [00:01<02:11,  2.76it/s]2026-04-26 07:10:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220206/demand.csv?_=1777201831
  1%|          | 4/365 [00:01<02:08,  2.81it/s]2026-04-26 07:10:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220207/demand.csv?_=1777201831
  1%|          | 5/365 [00:01<02:01,  2.97it/s]2026-04-26 07:10:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220208/demand.csv?_=1777201831
  2%|          | 6/365 [00:02<02:06,  2.85it/s]2026-04-26 07:10:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220209/demand.csv?_=1777201832
  2%|          | 7/365 [00:02<02:04,  2.88it/s]2026-04-26 07:10:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220210/demand.csv?_=1777201832
  2%|          | 8/365 [00:02<01:57,  3.04it/s]2026-04-26 07:10:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220211/demand.csv?_=1777201832
  2%|          | 9/365 [00:03<01:52,  3.15it/s]2026-04-26 07:10:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220212/demand.csv?_=1777201833
  3%|          | 10/365 [00:03<01:55,  3.07it/s]2026-04-26 07:10:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220213/demand.csv?_=1777201833
  3%|          | 11/365 [00:03<02:01,  2.91it/s]2026-04-26 07:10:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220214/demand.csv?_=1777201833
  3%|          | 12/365 [00:04<02:02,  2.87it/s]2026-04-26 07:10:34 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220215/demand.csv?_=1777201834
  4%|          | 13/365 [00:04<01:59,  2.94it/s]2026-04-26 07:10:34 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220216/demand.csv?_=1777201834
  4%|          | 14/365 [00:04<01:58,  2.95it/s]2026-04-26 07:10:35 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220217/demand.csv?_=1777201835
  4%|          | 15/365 [00:05<01:57,  2.98it/s]2026-04-26 07:10:35 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220218/demand.csv?_=1777201835
  4%|          | 16/365 [00:05<01:53,  3.08it/s]2026-04-26 07:10:35 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220219/demand.csv?_=1777201835
  5%|          | 17/365 [00:05<01:51,  3.12it/s]2026-04-26 07:10:35 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220220/demand.csv?_=1777201835
  5%|          | 18/365 [00:06<01:54,  3.02it/s]2026-04-26 07:10:36 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220221/demand.csv?_=1777201836
  5%|          | 19/365 [00:06<02:02,  2.82it/s]2026-04-26 07:10:36 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220222/demand.csv?_=1777201836
  5%|          | 20/365 [00:06<01:55,  2.98it/s]2026-04-26 07:10:36 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220223/demand.csv?_=1777201836
  6%|          | 21/365 [00:07<01:59,  2.89it/s]2026-04-26 07:10:37 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220224/demand.csv?_=1777201837
  6%|          | 22/365 [00:07<02:02,  2.80it/s]2026-04-26 07:10:37 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220225/demand.csv?_=1777201837
  6%|          | 23/365 [00:07<02:03,  2.77it/s]2026-04-26 07:10:38 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220226/demand.csv?_=1777201838
  7%|          | 24/365 [00:08<02:01,  2.81it/s]2026-04-26 07:10:38 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220227/demand.csv?_=1777201838
  7%|          | 25/365 [00:08<01:58,  2.87it/s]2026-04-26 07:10:38 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220228/demand.csv?_=1777201838
  7%|          | 26/365 [00:08<01:57,  2.89it/s]2026-04-26 07:10:39 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220301/demand.csv?_=1777201839
  7%|          | 27/365 [00:09<01:54,  2.95it/s]2026-04-26 07:10:39 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220302/demand.csv?_=1777201839
  8%|          | 28/365 [00:09<01:51,  3.01it/s]2026-04-26 07:10:39 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220303/demand.csv?_=1777201839
  8%|          | 29/365 [00:09<01:51,  3.01it/s]2026-04-26 07:10:40 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220304/demand.csv?_=1777201840
  8%|          | 30/365 [00:10<01:53,  2.96it/s]2026-04-26 07:10:40 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220305/demand.csv?_=1777201840
  8%|          | 31/365 [00:10<01:51,  2.99it/s]2026-04-26 07:10:40 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220306/demand.csv?_=1777201840
  9%|          | 32/365 [00:10<01:52,  2.95it/s]2026-04-26 07:10:41 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220307/demand.csv?_=1777201841
  9%|          | 33/365 [00:11<01:50,  3.01it/s]2026-04-26 07:10:41 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220308/demand.csv?_=1777201841
  9%|          | 34/365 [00:11<01:47,  3.07it/s]2026-04-26 07:10:41 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220309/demand.csv?_=1777201841
 10%|          | 35/365 [00:11<01:49,  3.02it/s]2026-04-26 07:10:42 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220310/demand.csv?_=1777201842
 10%|          | 36/365 [00:12<01:48,  3.04it/s]2026-04-26 07:10:42 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220311/demand.csv?_=1777201842
 10%|          | 37/365 [00:12<01:48,  3.03it/s]2026-04-26 07:10:42 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220312/demand.csv?_=1777201842
 10%|          | 38/365 [00:12<01:47,  3.03it/s]2026-04-26 07:10:43 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220313/demand.csv?_=1777201843
 11%|          | 39/365 [00:13<02:00,  2.70it/s]2026-04-26 07:10:43 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220314/demand.csv?_=1777201843
 11%|          | 40/365 [00:13<01:57,  2.76it/s]2026-04-26 07:10:43 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220315/demand.csv?_=1777201843
 11%|          | 41/365 [00:14<01:54,  2.83it/s]2026-04-26 07:10:44 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220316/demand.csv?_=1777201844
 12%|         | 42/365 [00:14<01:51,  2.89it/s]2026-04-26 07:10:44 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220317/demand.csv?_=1777201844
 12%|         | 43/365 [00:14<01:49,  2.95it/s]2026-04-26 07:10:44 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220318/demand.csv?_=1777201844
 12%|         | 44/365 [00:15<01:49,  2.94it/s]2026-04-26 07:10:45 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220319/demand.csv?_=1777201845
 12%|         | 45/365 [00:15<01:46,  3.01it/s]2026-04-26 07:10:45 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220320/demand.csv?_=1777201845
 13%|         | 46/365 [00:15<01:47,  2.96it/s]2026-04-26 07:10:45 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220321/demand.csv?_=1777201845
 13%|         | 47/365 [00:16<01:46,  2.97it/s]2026-04-26 07:10:46 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220322/demand.csv?_=1777201846
 13%|         | 48/365 [00:16<01:44,  3.03it/s]2026-04-26 07:10:46 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220323/demand.csv?_=1777201846
 13%|         | 49/365 [00:16<01:43,  3.05it/s]2026-04-26 07:10:46 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220324/demand.csv?_=1777201846
 14%|         | 50/365 [00:17<01:41,  3.12it/s]2026-04-26 07:10:47 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220325/demand.csv?_=1777201847
 14%|         | 51/365 [00:17<01:40,  3.13it/s]2026-04-26 07:10:47 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220326/demand.csv?_=1777201847
 14%|         | 52/365 [00:17<01:40,  3.13it/s]2026-04-26 07:10:47 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220327/demand.csv?_=1777201847
 15%|         | 53/365 [00:18<01:45,  2.95it/s]2026-04-26 07:10:48 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220328/demand.csv?_=1777201848
 15%|         | 54/365 [00:18<01:43,  2.99it/s]2026-04-26 07:10:48 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220329/demand.csv?_=1777201848
 15%|         | 55/365 [00:18<01:45,  2.93it/s]2026-04-26 07:10:48 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220330/demand.csv?_=1777201848
 15%|         | 56/365 [00:19<01:41,  3.05it/s]2026-04-26 07:10:49 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220331/demand.csv?_=1777201849
 16%|         | 57/365 [00:19<01:42,  3.00it/s]2026-04-26 07:10:49 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220401/demand.csv?_=1777201849
 16%|         | 58/365 [00:19<01:48,  2.84it/s]2026-04-26 07:10:49 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220402/demand.csv?_=1777201849
 16%|         | 59/365 [00:20<01:48,  2.82it/s]2026-04-26 07:10:50 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220403/demand.csv?_=1777201850
 16%|         | 60/365 [00:20<01:46,  2.87it/s]2026-04-26 07:10:50 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220404/demand.csv?_=1777201850
 17%|         | 61/365 [00:20<01:45,  2.89it/s]2026-04-26 07:10:50 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220405/demand.csv?_=1777201850
 17%|         | 62/365 [00:21<01:44,  2.89it/s]2026-04-26 07:10:51 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220406/demand.csv?_=1777201851
 17%|         | 63/365 [00:21<01:42,  2.93it/s]2026-04-26 07:10:51 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220407/demand.csv?_=1777201851
 18%|         | 64/365 [00:21<01:38,  3.05it/s]2026-04-26 07:10:51 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220408/demand.csv?_=1777201851
 18%|         | 65/365 [00:22<01:42,  2.92it/s]2026-04-26 07:10:52 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220409/demand.csv?_=1777201852
 18%|         | 66/365 [00:22<01:41,  2.93it/s]2026-04-26 07:10:52 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220410/demand.csv?_=1777201852
 18%|         | 67/365 [00:22<01:41,  2.93it/s]2026-04-26 07:10:52 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220411/demand.csv?_=1777201852
 19%|         | 68/365 [00:23<01:41,  2.94it/s]2026-04-26 07:10:53 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220412/demand.csv?_=1777201853
 19%|         | 69/365 [00:23<01:40,  2.94it/s]2026-04-26 07:10:53 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220413/demand.csv?_=1777201853
 19%|         | 70/365 [00:23<01:39,  2.97it/s]2026-04-26 07:10:53 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220414/demand.csv?_=1777201853
 19%|         | 71/365 [00:24<01:44,  2.83it/s]2026-04-26 07:10:54 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220415/demand.csv?_=1777201854
 20%|         | 72/365 [00:24<01:49,  2.68it/s]2026-04-26 07:10:54 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220416/demand.csv?_=1777201854
 20%|         | 73/365 [00:24<01:44,  2.80it/s]2026-04-26 07:10:55 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220417/demand.csv?_=1777201855
 20%|         | 74/365 [00:25<01:43,  2.82it/s]2026-04-26 07:10:55 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220418/demand.csv?_=1777201855
 21%|         | 75/365 [00:25<01:39,  2.93it/s]2026-04-26 07:10:55 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220419/demand.csv?_=1777201855
 21%|         | 76/365 [00:25<01:38,  2.94it/s]2026-04-26 07:10:56 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220420/demand.csv?_=1777201856
 21%|         | 77/365 [00:26<01:36,  2.99it/s]2026-04-26 07:10:56 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220421/demand.csv?_=1777201856
 21%|         | 78/365 [00:26<01:35,  3.00it/s]2026-04-26 07:10:56 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220422/demand.csv?_=1777201856
 22%|         | 79/365 [00:26<01:36,  2.95it/s]2026-04-26 07:10:57 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220423/demand.csv?_=1777201857
 22%|         | 80/365 [00:27<01:36,  2.95it/s]2026-04-26 07:10:57 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220424/demand.csv?_=1777201857
 22%|         | 81/365 [00:27<01:39,  2.86it/s]2026-04-26 07:10:57 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220425/demand.csv?_=1777201857
 22%|         | 82/365 [00:28<01:42,  2.76it/s]2026-04-26 07:10:58 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220426/demand.csv?_=1777201858
 23%|         | 83/365 [00:28<01:39,  2.85it/s]2026-04-26 07:10:58 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220427/demand.csv?_=1777201858
 23%|         | 84/365 [00:28<01:38,  2.85it/s]2026-04-26 07:10:58 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220428/demand.csv?_=1777201858
 23%|         | 85/365 [00:29<01:33,  3.00it/s]2026-04-26 07:10:59 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220429/demand.csv?_=1777201859
 24%|         | 86/365 [00:29<01:32,  3.02it/s]2026-04-26 07:10:59 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220430/demand.csv?_=1777201859
 24%|         | 87/365 [00:29<01:36,  2.87it/s]2026-04-26 07:10:59 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220501/demand.csv?_=1777201859
 24%|         | 88/365 [00:30<01:31,  3.01it/s]2026-04-26 07:11:00 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220502/demand.csv?_=1777201860
 24%|         | 89/365 [00:30<01:42,  2.70it/s]2026-04-26 07:11:00 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220503/demand.csv?_=1777201860
 25%|         | 90/365 [00:30<01:40,  2.74it/s]2026-04-26 07:11:01 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220504/demand.csv?_=1777201861
 25%|         | 91/365 [00:31<01:37,  2.81it/s]2026-04-26 07:11:01 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220505/demand.csv?_=1777201861
 25%|         | 92/365 [00:31<01:34,  2.90it/s]2026-04-26 07:11:01 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220506/demand.csv?_=1777201861
 25%|         | 93/365 [00:31<01:39,  2.73it/s]2026-04-26 07:11:02 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220507/demand.csv?_=1777201862
 26%|         | 94/365 [00:32<01:33,  2.88it/s]2026-04-26 07:11:02 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220508/demand.csv?_=1777201862
 26%|         | 95/365 [00:32<01:32,  2.91it/s]2026-04-26 07:11:02 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220509/demand.csv?_=1777201862
 26%|         | 96/365 [00:32<01:27,  3.08it/s]2026-04-26 07:11:02 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220510/demand.csv?_=1777201862
 27%|         | 97/365 [00:33<01:33,  2.88it/s]2026-04-26 07:11:03 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220511/demand.csv?_=1777201863
 27%|         | 98/365 [00:33<01:31,  2.91it/s]2026-04-26 07:11:03 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220512/demand.csv?_=1777201863
 27%|         | 99/365 [00:33<01:29,  2.99it/s]2026-04-26 07:11:04 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220513/demand.csv?_=1777201864
 27%|         | 100/365 [00:34<01:26,  3.07it/s]2026-04-26 07:11:04 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220514/demand.csv?_=1777201864
 28%|         | 101/365 [00:34<01:28,  3.00it/s]2026-04-26 07:11:04 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220515/demand.csv?_=1777201864
 28%|         | 102/365 [00:34<01:25,  3.06it/s]2026-04-26 07:11:05 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220516/demand.csv?_=1777201865
 28%|         | 103/365 [00:35<01:26,  3.02it/s]2026-04-26 07:11:05 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220517/demand.csv?_=1777201865
 28%|         | 104/365 [00:35<01:33,  2.81it/s]2026-04-26 07:11:05 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220518/demand.csv?_=1777201865
 29%|         | 105/365 [00:35<01:30,  2.88it/s]2026-04-26 07:11:06 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220519/demand.csv?_=1777201866
 29%|         | 106/365 [00:36<01:27,  2.96it/s]2026-04-26 07:11:06 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220520/demand.csv?_=1777201866
 29%|         | 107/365 [00:36<01:34,  2.73it/s]2026-04-26 07:11:06 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220521/demand.csv?_=1777201866
 30%|         | 108/365 [00:37<01:30,  2.84it/s]2026-04-26 07:11:07 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220522/demand.csv?_=1777201867
 30%|         | 109/365 [00:37<01:29,  2.88it/s]2026-04-26 07:11:07 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220523/demand.csv?_=1777201867
 30%|         | 110/365 [00:37<01:27,  2.91it/s]2026-04-26 07:11:07 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220524/demand.csv?_=1777201867
 30%|         | 111/365 [00:38<01:27,  2.91it/s]2026-04-26 07:11:08 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220525/demand.csv?_=1777201868
 31%|         | 112/365 [00:38<01:24,  2.98it/s]2026-04-26 07:11:08 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220526/demand.csv?_=1777201868
 31%|         | 113/365 [00:38<01:21,  3.08it/s]2026-04-26 07:11:08 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220527/demand.csv?_=1777201868
 31%|         | 114/365 [00:38<01:22,  3.05it/s]2026-04-26 07:11:09 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220528/demand.csv?_=1777201869
 32%|         | 115/365 [00:39<01:22,  3.04it/s]2026-04-26 07:11:09 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220529/demand.csv?_=1777201869
 32%|         | 116/365 [00:39<01:23,  2.96it/s]2026-04-26 07:11:09 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220530/demand.csv?_=1777201869
 32%|         | 117/365 [00:39<01:22,  2.99it/s]2026-04-26 07:11:10 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220531/demand.csv?_=1777201870
 32%|         | 118/365 [00:40<01:22,  3.00it/s]2026-04-26 07:11:10 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220601/demand.csv?_=1777201870
 33%|         | 119/365 [00:40<01:22,  2.99it/s]2026-04-26 07:11:10 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220602/demand.csv?_=1777201870
 33%|         | 120/365 [00:40<01:19,  3.09it/s]2026-04-26 07:11:11 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220603/demand.csv?_=1777201871
 33%|         | 121/365 [00:41<01:22,  2.97it/s]2026-04-26 07:11:11 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220604/demand.csv?_=1777201871
 33%|         | 122/365 [00:41<01:20,  3.02it/s]2026-04-26 07:11:11 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220605/demand.csv?_=1777201871
 34%|         | 123/365 [00:41<01:19,  3.06it/s]2026-04-26 07:11:12 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220606/demand.csv?_=1777201872
 34%|         | 124/365 [00:42<01:19,  3.04it/s]2026-04-26 07:11:12 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220607/demand.csv?_=1777201872
 34%|         | 125/365 [00:42<01:19,  3.02it/s]2026-04-26 07:11:12 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220608/demand.csv?_=1777201872
 35%|         | 126/365 [00:42<01:17,  3.08it/s]2026-04-26 07:11:13 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220609/demand.csv?_=1777201873
 35%|         | 127/365 [00:43<01:17,  3.07it/s]2026-04-26 07:11:13 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220610/demand.csv?_=1777201873
 35%|         | 128/365 [00:43<01:17,  3.07it/s]2026-04-26 07:11:13 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220611/demand.csv?_=1777201873
 35%|        | 129/365 [00:43<01:17,  3.05it/s]2026-04-26 07:11:14 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220612/demand.csv?_=1777201874
 36%|        | 130/365 [00:44<01:19,  2.94it/s]2026-04-26 07:11:14 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220613/demand.csv?_=1777201874
 36%|        | 131/365 [00:44<01:22,  2.84it/s]2026-04-26 07:11:14 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220614/demand.csv?_=1777201874
 36%|        | 132/365 [00:45<01:23,  2.77it/s]2026-04-26 07:11:15 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220615/demand.csv?_=1777201875
 36%|        | 133/365 [00:45<01:24,  2.76it/s]2026-04-26 07:11:15 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220616/demand.csv?_=1777201875
 37%|        | 134/365 [00:45<01:25,  2.70it/s]2026-04-26 07:11:15 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220617/demand.csv?_=1777201875
 37%|        | 135/365 [00:46<01:24,  2.73it/s]2026-04-26 07:11:16 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220618/demand.csv?_=1777201876
 37%|        | 136/365 [00:46<01:19,  2.86it/s]2026-04-26 07:11:16 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220619/demand.csv?_=1777201876
 38%|        | 137/365 [00:46<01:17,  2.95it/s]2026-04-26 07:11:16 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220620/demand.csv?_=1777201876
 38%|        | 138/365 [00:47<01:16,  2.96it/s]2026-04-26 07:11:17 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220621/demand.csv?_=1777201877
 38%|        | 139/365 [00:47<01:16,  2.97it/s]2026-04-26 07:11:17 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220622/demand.csv?_=1777201877
 38%|        | 140/365 [00:47<01:15,  2.97it/s]2026-04-26 07:11:17 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220623/demand.csv?_=1777201877
 39%|        | 141/365 [00:48<01:14,  3.00it/s]2026-04-26 07:11:18 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220624/demand.csv?_=1777201878
 39%|        | 142/365 [00:48<01:12,  3.08it/s]2026-04-26 07:11:18 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220625/demand.csv?_=1777201878
 39%|        | 143/365 [00:48<01:11,  3.09it/s]2026-04-26 07:11:18 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220626/demand.csv?_=1777201878
 39%|        | 144/365 [00:49<01:08,  3.20it/s]2026-04-26 07:11:19 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220627/demand.csv?_=1777201879
 40%|        | 145/365 [00:49<01:14,  2.96it/s]2026-04-26 07:11:19 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220628/demand.csv?_=1777201879
 40%|        | 146/365 [00:49<01:12,  3.03it/s]2026-04-26 07:11:19 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220629/demand.csv?_=1777201879
 40%|        | 147/365 [00:50<01:13,  2.95it/s]2026-04-26 07:11:20 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220630/demand.csv?_=1777201880
 41%|        | 148/365 [00:50<01:14,  2.92it/s]2026-04-26 07:11:20 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220701/demand.csv?_=1777201880
 41%|        | 149/365 [00:50<01:12,  2.97it/s]2026-04-26 07:11:20 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220702/demand.csv?_=1777201880
 41%|        | 150/365 [00:51<01:13,  2.91it/s]2026-04-26 07:11:21 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220703/demand.csv?_=1777201881
 41%|        | 151/365 [00:51<01:13,  2.90it/s]2026-04-26 07:11:21 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220704/demand.csv?_=1777201881
 42%|        | 152/365 [00:51<01:13,  2.90it/s]2026-04-26 07:11:21 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220705/demand.csv?_=1777201881
 42%|        | 153/365 [00:52<01:11,  2.95it/s]2026-04-26 07:11:22 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220706/demand.csv?_=1777201882
 42%|        | 154/365 [00:52<01:13,  2.85it/s]2026-04-26 07:11:22 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220707/demand.csv?_=1777201882
 42%|        | 155/365 [00:52<01:14,  2.83it/s]2026-04-26 07:11:23 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220708/demand.csv?_=1777201883
 43%|        | 156/365 [00:53<01:11,  2.91it/s]2026-04-26 07:11:23 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220709/demand.csv?_=1777201883
 43%|        | 157/365 [00:53<01:10,  2.93it/s]2026-04-26 07:11:23 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220710/demand.csv?_=1777201883
 43%|        | 158/365 [00:53<01:10,  2.92it/s]2026-04-26 07:11:24 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220711/demand.csv?_=1777201884
 44%|        | 159/365 [00:54<01:10,  2.94it/s]2026-04-26 07:11:24 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220712/demand.csv?_=1777201884
 44%|        | 160/365 [00:54<01:08,  3.00it/s]2026-04-26 07:11:24 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220713/demand.csv?_=1777201884
 44%|        | 161/365 [00:54<01:07,  3.03it/s]2026-04-26 07:11:25 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220714/demand.csv?_=1777201885
 44%|        | 162/365 [00:55<01:08,  2.98it/s]2026-04-26 07:11:25 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220715/demand.csv?_=1777201885
 45%|        | 163/365 [00:55<01:06,  3.02it/s]2026-04-26 07:11:25 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220716/demand.csv?_=1777201885
 45%|        | 164/365 [00:55<01:06,  3.04it/s]2026-04-26 07:11:26 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220717/demand.csv?_=1777201886
 45%|        | 165/365 [00:56<01:08,  2.94it/s]2026-04-26 07:11:26 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220718/demand.csv?_=1777201886
 45%|        | 166/365 [00:56<01:08,  2.91it/s]2026-04-26 07:11:26 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220719/demand.csv?_=1777201886
 46%|        | 167/365 [00:56<01:07,  2.95it/s]2026-04-26 07:11:27 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220720/demand.csv?_=1777201887
 46%|        | 168/365 [00:57<01:06,  2.95it/s]2026-04-26 07:11:27 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220721/demand.csv?_=1777201887
 46%|        | 169/365 [00:57<01:07,  2.90it/s]2026-04-26 07:11:27 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220722/demand.csv?_=1777201887
 47%|        | 170/365 [00:57<01:06,  2.92it/s]2026-04-26 07:11:28 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220723/demand.csv?_=1777201888
 47%|        | 171/365 [00:58<01:05,  2.98it/s]2026-04-26 07:11:28 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220724/demand.csv?_=1777201888
 47%|        | 172/365 [00:58<01:08,  2.82it/s]2026-04-26 07:11:28 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220725/demand.csv?_=1777201888
 47%|        | 173/365 [00:58<01:05,  2.91it/s]2026-04-26 07:11:29 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220726/demand.csv?_=1777201889
 48%|        | 174/365 [00:59<01:05,  2.94it/s]2026-04-26 07:11:29 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220727/demand.csv?_=1777201889
 48%|        | 175/365 [00:59<01:03,  3.00it/s]2026-04-26 07:11:29 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220728/demand.csv?_=1777201889
 48%|        | 176/365 [00:59<01:03,  2.99it/s]2026-04-26 07:11:30 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220729/demand.csv?_=1777201890
 48%|        | 177/365 [01:00<01:02,  3.00it/s]2026-04-26 07:11:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220730/demand.csv?_=1777201890
 49%|        | 178/365 [01:00<01:08,  2.72it/s]2026-04-26 07:11:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220731/demand.csv?_=1777201890
 49%|        | 179/365 [01:01<01:04,  2.87it/s]2026-04-26 07:11:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220801/demand.csv?_=1777201891
 49%|        | 180/365 [01:01<01:02,  2.95it/s]2026-04-26 07:11:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220802/demand.csv?_=1777201891
 50%|        | 181/365 [01:01<01:02,  2.93it/s]2026-04-26 07:11:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220803/demand.csv?_=1777201891
 50%|        | 182/365 [01:02<01:02,  2.91it/s]2026-04-26 07:11:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220804/demand.csv?_=1777201892
 50%|        | 183/365 [01:02<01:01,  2.94it/s]2026-04-26 07:11:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220805/demand.csv?_=1777201892
 50%|        | 184/365 [01:02<01:03,  2.86it/s]2026-04-26 07:11:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220806/demand.csv?_=1777201892
 51%|        | 185/365 [01:03<01:01,  2.95it/s]2026-04-26 07:11:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220807/demand.csv?_=1777201893
 51%|        | 186/365 [01:03<01:02,  2.88it/s]2026-04-26 07:11:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220808/demand.csv?_=1777201893
 51%|        | 187/365 [01:03<01:01,  2.89it/s]2026-04-26 07:11:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220809/demand.csv?_=1777201893
 52%|        | 188/365 [01:04<01:00,  2.94it/s]2026-04-26 07:11:34 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220810/demand.csv?_=1777201894
 52%|        | 189/365 [01:04<00:59,  2.95it/s]2026-04-26 07:11:34 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220811/demand.csv?_=1777201894
 52%|        | 190/365 [01:04<01:00,  2.91it/s]2026-04-26 07:11:34 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220812/demand.csv?_=1777201894
 52%|        | 191/365 [01:05<00:59,  2.94it/s]2026-04-26 07:11:35 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220813/demand.csv?_=1777201895
 53%|        | 192/365 [01:05<00:58,  2.97it/s]2026-04-26 07:11:35 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220814/demand.csv?_=1777201895
 53%|       | 193/365 [01:05<01:01,  2.78it/s]2026-04-26 07:11:36 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220815/demand.csv?_=1777201896
 53%|       | 194/365 [01:06<01:02,  2.73it/s]2026-04-26 07:11:36 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220816/demand.csv?_=1777201896
 53%|       | 195/365 [01:06<01:02,  2.71it/s]2026-04-26 07:11:36 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220817/demand.csv?_=1777201896
 54%|       | 196/365 [01:06<01:00,  2.81it/s]2026-04-26 07:11:37 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220818/demand.csv?_=1777201897
 54%|       | 197/365 [01:07<00:58,  2.85it/s]2026-04-26 07:11:37 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220819/demand.csv?_=1777201897
 54%|       | 198/365 [01:07<00:57,  2.91it/s]2026-04-26 07:11:37 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220820/demand.csv?_=1777201897
 55%|       | 199/365 [01:07<00:56,  2.95it/s]2026-04-26 07:11:38 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220821/demand.csv?_=1777201898
 55%|       | 200/365 [01:08<00:55,  2.99it/s]2026-04-26 07:11:38 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220822/demand.csv?_=1777201898
 55%|       | 201/365 [01:08<00:56,  2.93it/s]2026-04-26 07:11:38 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220823/demand.csv?_=1777201898
 55%|       | 202/365 [01:08<00:54,  2.99it/s]2026-04-26 07:11:39 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220824/demand.csv?_=1777201899
 56%|       | 203/365 [01:09<00:52,  3.09it/s]2026-04-26 07:11:39 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220825/demand.csv?_=1777201899
 56%|       | 204/365 [01:09<00:52,  3.04it/s]2026-04-26 07:11:39 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220826/demand.csv?_=1777201899
 56%|       | 205/365 [01:09<00:50,  3.16it/s]2026-04-26 07:11:40 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220827/demand.csv?_=1777201900
 56%|       | 206/365 [01:10<00:49,  3.22it/s]2026-04-26 07:11:40 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220828/demand.csv?_=1777201900
 57%|       | 207/365 [01:10<00:49,  3.20it/s]2026-04-26 07:11:40 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220829/demand.csv?_=1777201900
 57%|       | 208/365 [01:10<00:50,  3.11it/s]2026-04-26 07:11:40 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220830/demand.csv?_=1777201900
 57%|       | 209/365 [01:11<00:48,  3.21it/s]2026-04-26 07:11:41 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220831/demand.csv?_=1777201901
 58%|       | 210/365 [01:11<00:49,  3.11it/s]2026-04-26 07:11:41 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220901/demand.csv?_=1777201901
 58%|       | 211/365 [01:11<00:49,  3.13it/s]2026-04-26 07:11:41 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220902/demand.csv?_=1777201901
 58%|       | 212/365 [01:12<00:47,  3.20it/s]2026-04-26 07:11:42 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220903/demand.csv?_=1777201902
 58%|       | 213/365 [01:12<00:48,  3.13it/s]2026-04-26 07:11:42 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220904/demand.csv?_=1777201902
 59%|       | 214/365 [01:12<00:47,  3.16it/s]2026-04-26 07:11:42 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220905/demand.csv?_=1777201902
 59%|       | 215/365 [01:13<00:47,  3.16it/s]2026-04-26 07:11:43 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220906/demand.csv?_=1777201903
 59%|       | 216/365 [01:13<00:48,  3.07it/s]2026-04-26 07:11:43 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220907/demand.csv?_=1777201903
 59%|       | 217/365 [01:13<00:48,  3.05it/s]2026-04-26 07:11:43 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220908/demand.csv?_=1777201903
 60%|       | 218/365 [01:14<00:48,  3.03it/s]2026-04-26 07:11:44 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220909/demand.csv?_=1777201904
 60%|       | 219/365 [01:14<00:49,  2.97it/s]2026-04-26 07:11:44 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220910/demand.csv?_=1777201904
 60%|       | 220/365 [01:14<00:47,  3.04it/s]2026-04-26 07:11:44 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220911/demand.csv?_=1777201904
 61%|       | 221/365 [01:15<00:48,  2.95it/s]2026-04-26 07:11:45 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220912/demand.csv?_=1777201905
 61%|       | 222/365 [01:15<00:48,  2.95it/s]2026-04-26 07:11:45 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220913/demand.csv?_=1777201905
 61%|       | 223/365 [01:15<00:47,  3.00it/s]2026-04-26 07:11:45 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220914/demand.csv?_=1777201905
 61%|       | 224/365 [01:16<00:47,  2.96it/s]2026-04-26 07:11:46 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20220915/demand.csv?_=1777201906
 62%|       | 225/365 [01:16<00:47,  2.96it/s]2026-04-26 07:11:46 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220916/demand.csv?_=1777201906
 62%|       | 226/365 [01:16<00:46,  2.97it/s]2026-04-26 07:11:46 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220917/demand.csv?_=1777201906
 62%|       | 227/365 [01:17<00:46,  2.99it/s]2026-04-26 07:11:47 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220918/demand.csv?_=1777201907
 62%|       | 228/365 [01:17<00:47,  2.90it/s]2026-04-26 07:11:47 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220919/demand.csv?_=1777201907
 63%|       | 229/365 [01:17<00:46,  2.94it/s]2026-04-26 07:11:47 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220920/demand.csv?_=1777201907
 63%|       | 230/365 [01:18<00:46,  2.92it/s]2026-04-26 07:11:48 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220921/demand.csv?_=1777201908
 63%|       | 231/365 [01:18<00:44,  3.02it/s]2026-04-26 07:11:48 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220922/demand.csv?_=1777201908
 64%|       | 232/365 [01:18<00:42,  3.13it/s]2026-04-26 07:11:48 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220923/demand.csv?_=1777201908
 64%|       | 233/365 [01:19<00:43,  3.02it/s]2026-04-26 07:11:49 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220924/demand.csv?_=1777201909
 64%|       | 234/365 [01:19<00:42,  3.11it/s]2026-04-26 07:11:49 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220925/demand.csv?_=1777201909
 64%|       | 235/365 [01:19<00:40,  3.17it/s]2026-04-26 07:11:49 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220926/demand.csv?_=1777201909
 65%|       | 236/365 [01:20<00:42,  3.04it/s]2026-04-26 07:11:50 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220927/demand.csv?_=1777201910
 65%|       | 237/365 [01:20<00:40,  3.16it/s]2026-04-26 07:11:50 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220928/demand.csv?_=1777201910
 65%|       | 238/365 [01:20<00:40,  3.11it/s]2026-04-26 07:11:50 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220929/demand.csv?_=1777201910
 65%|       | 239/365 [01:21<00:42,  2.98it/s]2026-04-26 07:11:51 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20220930/demand.csv?_=1777201911
 66%|       | 240/365 [01:21<00:41,  3.03it/s]2026-04-26 07:11:51 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20221001/demand.csv?_=1777201911
 66%|       | 241/365 [01:21<00:41,  2.98it/s]2026-04-26 07:11:51 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221002/demand.csv?_=1777201911
 66%|       | 242/365 [01:22<00:43,  2.82it/s]2026-04-26 07:11:52 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221003/demand.csv?_=1777201912
 67%|       | 243/365 [01:22<00:42,  2.88it/s]2026-04-26 07:11:52 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221004/demand.csv?_=1777201912
 67%|       | 244/365 [01:22<00:40,  3.00it/s]2026-04-26 07:11:52 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221005/demand.csv?_=1777201912
 67%|       | 245/365 [01:23<00:40,  2.99it/s]2026-04-26 07:11:53 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221006/demand.csv?_=1777201913
 67%|       | 246/365 [01:23<00:40,  2.93it/s]2026-04-26 07:11:53 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221007/demand.csv?_=1777201913
 68%|       | 247/365 [01:23<00:39,  3.01it/s]2026-04-26 07:11:53 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221008/demand.csv?_=1777201913
 68%|       | 248/365 [01:24<00:37,  3.09it/s]2026-04-26 07:11:54 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221009/demand.csv?_=1777201914
 68%|       | 249/365 [01:24<00:38,  3.04it/s]2026-04-26 07:11:54 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221010/demand.csv?_=1777201914
 68%|       | 250/365 [01:24<00:37,  3.05it/s]2026-04-26 07:11:54 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221011/demand.csv?_=1777201914
 69%|       | 251/365 [01:25<00:38,  2.97it/s]2026-04-26 07:11:55 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221012/demand.csv?_=1777201915
 69%|       | 252/365 [01:25<00:37,  2.99it/s]2026-04-26 07:11:55 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221013/demand.csv?_=1777201915
 69%|       | 253/365 [01:25<00:38,  2.93it/s]2026-04-26 07:11:55 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221014/demand.csv?_=1777201915
 70%|       | 254/365 [01:26<00:37,  2.98it/s]2026-04-26 07:11:56 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221015/demand.csv?_=1777201916
 70%|       | 255/365 [01:26<00:36,  3.00it/s]2026-04-26 07:11:56 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221016/demand.csv?_=1777201916
 70%|       | 256/365 [01:26<00:36,  2.95it/s]2026-04-26 07:11:56 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20221017/demand.csv?_=1777201916
 70%|       | 257/365 [01:27<00:37,  2.84it/s]2026-04-26 07:11:57 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221018/demand.csv?_=1777201917
 71%|       | 258/365 [01:27<00:37,  2.89it/s]2026-04-26 07:11:57 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221019/demand.csv?_=1777201917
 71%|       | 259/365 [01:27<00:36,  2.93it/s]2026-04-26 07:11:57 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221020/demand.csv?_=1777201917
 71%|       | 260/365 [01:28<00:35,  2.98it/s]2026-04-26 07:11:58 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221021/demand.csv?_=1777201918
 72%|      | 261/365 [01:28<00:34,  3.00it/s]2026-04-26 07:11:58 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221022/demand.csv?_=1777201918
 72%|      | 262/365 [01:28<00:34,  2.98it/s]2026-04-26 07:11:58 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221023/demand.csv?_=1777201918
 72%|      | 263/365 [01:29<00:34,  2.98it/s]2026-04-26 07:11:59 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221024/demand.csv?_=1777201919
 72%|      | 264/365 [01:29<00:33,  3.00it/s]2026-04-26 07:11:59 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221025/demand.csv?_=1777201919
 73%|      | 265/365 [01:29<00:32,  3.03it/s]2026-04-26 07:11:59 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221026/demand.csv?_=1777201919
 73%|      | 266/365 [01:30<00:32,  3.00it/s]2026-04-26 07:12:00 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221027/demand.csv?_=1777201920
 73%|      | 267/365 [01:30<00:34,  2.83it/s]2026-04-26 07:12:00 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221028/demand.csv?_=1777201920
 73%|      | 268/365 [01:30<00:34,  2.83it/s]2026-04-26 07:12:01 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221029/demand.csv?_=1777201921
 74%|      | 269/365 [01:31<00:33,  2.88it/s]2026-04-26 07:12:01 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221030/demand.csv?_=1777201921
 74%|      | 270/365 [01:31<00:33,  2.83it/s]2026-04-26 07:12:01 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221031/demand.csv?_=1777201921
 74%|      | 271/365 [01:31<00:33,  2.80it/s]2026-04-26 07:12:02 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221101/demand.csv?_=1777201922
 75%|      | 272/365 [01:32<00:32,  2.85it/s]2026-04-26 07:12:02 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20221102/demand.csv?_=1777201922
 75%|      | 273/365 [01:32<00:31,  2.90it/s]2026-04-26 07:12:02 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221103/demand.csv?_=1777201922
 75%|      | 274/365 [01:32<00:31,  2.89it/s]2026-04-26 07:12:03 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221104/demand.csv?_=1777201923
 75%|      | 275/365 [01:33<00:30,  2.96it/s]2026-04-26 07:12:03 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221105/demand.csv?_=1777201923
 76%|      | 276/365 [01:33<00:29,  3.07it/s]2026-04-26 07:12:03 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221106/demand.csv?_=1777201923
 76%|      | 277/365 [01:33<00:28,  3.05it/s]2026-04-26 07:12:04 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221107/demand.csv?_=1777201924
 76%|      | 278/365 [01:34<00:28,  3.03it/s]2026-04-26 07:12:04 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221108/demand.csv?_=1777201924
 76%|      | 279/365 [01:34<00:28,  3.06it/s]2026-04-26 07:12:04 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221109/demand.csv?_=1777201924
 77%|      | 280/365 [01:34<00:28,  3.02it/s]2026-04-26 07:12:05 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221110/demand.csv?_=1777201925
 77%|      | 281/365 [01:35<00:27,  3.03it/s]2026-04-26 07:12:05 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221111/demand.csv?_=1777201925
 77%|      | 282/365 [01:35<00:28,  2.95it/s]2026-04-26 07:12:05 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221112/demand.csv?_=1777201925
 78%|      | 283/365 [01:35<00:29,  2.80it/s]2026-04-26 07:12:06 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221113/demand.csv?_=1777201926
 78%|      | 284/365 [01:36<00:28,  2.85it/s]2026-04-26 07:12:06 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221114/demand.csv?_=1777201926
 78%|      | 285/365 [01:36<00:28,  2.82it/s]2026-04-26 07:12:06 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221115/demand.csv?_=1777201926
 78%|      | 286/365 [01:37<00:27,  2.84it/s]2026-04-26 07:12:07 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221116/demand.csv?_=1777201927
 79%|      | 287/365 [01:37<00:27,  2.87it/s]2026-04-26 07:12:07 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221117/demand.csv?_=1777201927
 79%|      | 288/365 [01:37<00:25,  3.00it/s]2026-04-26 07:12:07 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20221118/demand.csv?_=1777201927
 79%|      | 289/365 [01:38<00:27,  2.78it/s]2026-04-26 07:12:08 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221119/demand.csv?_=1777201928
 79%|      | 290/365 [01:38<00:26,  2.82it/s]2026-04-26 07:12:08 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221120/demand.csv?_=1777201928
 80%|      | 291/365 [01:38<00:26,  2.83it/s]2026-04-26 07:12:08 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221121/demand.csv?_=1777201928
 80%|      | 292/365 [01:39<00:25,  2.86it/s]2026-04-26 07:12:09 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221122/demand.csv?_=1777201929
 80%|      | 293/365 [01:39<00:24,  2.91it/s]2026-04-26 07:12:09 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221123/demand.csv?_=1777201929
 81%|      | 294/365 [01:39<00:23,  2.96it/s]2026-04-26 07:12:09 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221124/demand.csv?_=1777201929
 81%|      | 295/365 [01:40<00:23,  3.04it/s]2026-04-26 07:12:10 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221125/demand.csv?_=1777201930
 81%|      | 296/365 [01:40<00:23,  2.98it/s]2026-04-26 07:12:10 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221126/demand.csv?_=1777201930
 81%|      | 297/365 [01:40<00:22,  2.97it/s]2026-04-26 07:12:10 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221127/demand.csv?_=1777201930
 82%|      | 298/365 [01:41<00:21,  3.05it/s]2026-04-26 07:12:11 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221128/demand.csv?_=1777201931
 82%|      | 299/365 [01:41<00:22,  2.98it/s]2026-04-26 07:12:11 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221129/demand.csv?_=1777201931
 82%|      | 300/365 [01:41<00:21,  2.98it/s]2026-04-26 07:12:11 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221130/demand.csv?_=1777201931
 82%|      | 301/365 [01:42<00:21,  3.03it/s]2026-04-26 07:12:12 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221201/demand.csv?_=1777201932
 83%|      | 302/365 [01:42<00:20,  3.03it/s]2026-04-26 07:12:12 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221202/demand.csv?_=1777201932
 83%|      | 303/365 [01:42<00:20,  3.09it/s]2026-04-26 07:12:12 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221203/demand.csv?_=1777201932
 83%|      | 304/365 [01:43<00:20,  3.05it/s]2026-04-26 07:12:13 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20221204/demand.csv?_=1777201933
 84%|      | 305/365 [01:43<00:19,  3.01it/s]2026-04-26 07:12:13 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221205/demand.csv?_=1777201933
 84%|      | 306/365 [01:43<00:20,  2.93it/s]2026-04-26 07:12:13 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221206/demand.csv?_=1777201933
 84%|      | 307/365 [01:44<00:19,  2.96it/s]2026-04-26 07:12:14 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221207/demand.csv?_=1777201934
 84%|      | 308/365 [01:44<00:19,  2.96it/s]2026-04-26 07:12:14 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221208/demand.csv?_=1777201934
 85%|      | 309/365 [01:44<00:18,  3.03it/s]2026-04-26 07:12:14 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221209/demand.csv?_=1777201934
 85%|      | 310/365 [01:45<00:18,  3.02it/s]2026-04-26 07:12:15 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221210/demand.csv?_=1777201935
 85%|      | 311/365 [01:45<00:17,  3.03it/s]2026-04-26 07:12:15 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221211/demand.csv?_=1777201935
 85%|      | 312/365 [01:45<00:17,  3.00it/s]2026-04-26 07:12:15 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221212/demand.csv?_=1777201935
 86%|      | 313/365 [01:46<00:17,  2.92it/s]2026-04-26 07:12:16 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221213/demand.csv?_=1777201936
 86%|      | 314/365 [01:46<00:17,  3.00it/s]2026-04-26 07:12:16 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221214/demand.csv?_=1777201936
 86%|      | 315/365 [01:46<00:16,  3.08it/s]2026-04-26 07:12:16 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221215/demand.csv?_=1777201936
 87%|      | 316/365 [01:47<00:15,  3.11it/s]2026-04-26 07:12:17 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221216/demand.csv?_=1777201937
 87%|      | 317/365 [01:47<00:15,  3.03it/s]2026-04-26 07:12:17 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221217/demand.csv?_=1777201937
 87%|      | 318/365 [01:47<00:15,  2.99it/s]2026-04-26 07:12:17 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221218/demand.csv?_=1777201937
 87%|      | 319/365 [01:48<00:15,  3.06it/s]2026-04-26 07:12:18 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221219/demand.csv?_=1777201938
 88%|      | 320/365 [01:48<00:16,  2.69it/s]2026-04-26 07:12:18 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20221220/demand.csv?_=1777201938
 88%|      | 321/365 [01:48<00:15,  2.80it/s]2026-04-26 07:12:19 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221221/demand.csv?_=1777201939
 88%|      | 322/365 [01:49<00:15,  2.83it/s]2026-04-26 07:12:19 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221222/demand.csv?_=1777201939
 88%|      | 323/365 [01:49<00:14,  2.84it/s]2026-04-26 07:12:19 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221223/demand.csv?_=1777201939
 89%|      | 324/365 [01:49<00:14,  2.77it/s]2026-04-26 07:12:20 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221224/demand.csv?_=1777201940
 89%|      | 325/365 [01:50<00:14,  2.81it/s]2026-04-26 07:12:20 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221225/demand.csv?_=1777201940
 89%|      | 326/365 [01:50<00:13,  2.83it/s]2026-04-26 07:12:20 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221226/demand.csv?_=1777201940
 90%|      | 327/365 [01:50<00:13,  2.85it/s]2026-04-26 07:12:21 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221227/demand.csv?_=1777201941
 90%|      | 328/365 [01:51<00:12,  2.88it/s]2026-04-26 07:12:21 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221228/demand.csv?_=1777201941
 90%|      | 329/365 [01:51<00:12,  2.83it/s]2026-04-26 07:12:21 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221229/demand.csv?_=1777201941
 90%|      | 330/365 [01:52<00:12,  2.88it/s]2026-04-26 07:12:22 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221230/demand.csv?_=1777201942
 91%|      | 331/365 [01:52<00:11,  2.96it/s]2026-04-26 07:12:22 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20221231/demand.csv?_=1777201942
 91%|      | 332/365 [01:52<00:11,  2.84it/s]2026-04-26 07:12:22 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230101/demand.csv?_=1777201942
 91%|      | 333/365 [01:53<00:10,  2.93it/s]2026-04-26 07:12:23 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230102/demand.csv?_=1777201943
 92%|      | 334/365 [01:53<00:10,  2.91it/s]2026-04-26 07:12:23 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230103/demand.csv?_=1777201943
 92%|      | 335/365 [01:53<00:10,  2.85it/s]2026-04-26 07:12:23 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230104/demand.csv?_=1777201943
 92%|      | 336/365 [01:54<00:09,  2.91it/s]2026-04-26 07:12:24 - INFO -
```

```
Fetching URL:
https://www.caiso.com/outlook/history/20230105/demand.csv?_=1777201944
 92%|      | 337/365 [01:54<00:09,  2.90it/s]2026-04-26 07:12:24 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230106/demand.csv?_=1777201944
 93%|      | 338/365 [01:54<00:08,  3.01it/s]2026-04-26 07:12:24 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230107/demand.csv?_=1777201944
 93%|      | 339/365 [01:55<00:08,  2.99it/s]2026-04-26 07:12:25 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230108/demand.csv?_=1777201945
 93%|      | 340/365 [01:55<00:08,  3.04it/s]2026-04-26 07:12:25 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230109/demand.csv?_=1777201945
 93%|      | 341/365 [01:55<00:07,  3.12it/s]2026-04-26 07:12:25 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230110/demand.csv?_=1777201945
 94%|      | 342/365 [01:55<00:07,  3.13it/s]2026-04-26 07:12:26 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230111/demand.csv?_=1777201946
 94%|      | 343/365 [01:56<00:07,  3.14it/s]2026-04-26 07:12:26 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230112/demand.csv?_=1777201946
 94%|      | 344/365 [01:56<00:06,  3.05it/s]2026-04-26 07:12:26 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230113/demand.csv?_=1777201946
 95%|      | 345/365 [01:56<00:06,  3.08it/s]2026-04-26 07:12:27 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230114/demand.csv?_=1777201947
 95%|      | 346/365 [01:57<00:06,  3.01it/s]2026-04-26 07:12:27 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230115/demand.csv?_=1777201947
 95%|      | 347/365 [01:57<00:05,  3.10it/s]2026-04-26 07:12:27 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230116/demand.csv?_=1777201947
 95%|      | 348/365 [01:57<00:05,  3.17it/s]2026-04-26 07:12:28 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230117/demand.csv?_=1777201948
 96%|      | 349/365 [01:58<00:05,  3.08it/s]2026-04-26 07:12:28 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230118/demand.csv?_=1777201948
 96%|      | 350/365 [01:58<00:04,  3.09it/s]2026-04-26 07:12:28 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230119/demand.csv?_=1777201948
 96%|      | 351/365 [01:58<00:04,  3.06it/s]2026-04-26 07:12:29 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230120/demand.csv?_=1777201949
 96%|      | 352/365 [01:59<00:04,  3.03it/s]2026-04-26 07:12:29 - INFO -
```

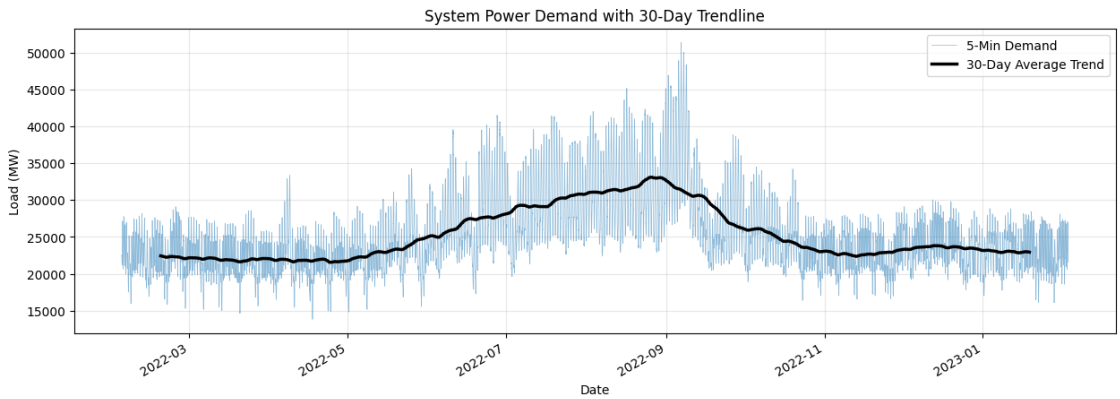
```
Fetching URL:
https://www.caiso.com/outlook/history/20230121/demand.csv?_=1777201949
 97%|      | 353/365 [01:59<00:03,  3.08it/s]2026-04-26 07:12:29 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230122/demand.csv?_=1777201949
 97%|      | 354/365 [01:59<00:03,  2.90it/s]2026-04-26 07:12:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230123/demand.csv?_=1777201950
 97%|      | 355/365 [02:00<00:03,  3.04it/s]2026-04-26 07:12:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230124/demand.csv?_=1777201950
 98%|      | 356/365 [02:00<00:03,  2.82it/s]2026-04-26 07:12:30 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230125/demand.csv?_=1777201950
 98%|      | 357/365 [02:01<00:02,  2.85it/s]2026-04-26 07:12:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230126/demand.csv?_=1777201951
 98%|      | 358/365 [02:01<00:02,  2.79it/s]2026-04-26 07:12:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230127/demand.csv?_=1777201951
 98%|      | 359/365 [02:01<00:02,  2.89it/s]2026-04-26 07:12:31 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230128/demand.csv?_=1777201951
 99%|      | 360/365 [02:02<00:01,  2.88it/s]2026-04-26 07:12:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230129/demand.csv?_=1777201952
 99%|      | 361/365 [02:02<00:01,  2.92it/s]2026-04-26 07:12:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230130/demand.csv?_=1777201952
 99%|      | 362/365 [02:02<00:01,  2.96it/s]2026-04-26 07:12:32 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230131/demand.csv?_=1777201952
 99%|      | 363/365 [02:03<00:00,  2.99it/s]2026-04-26 07:12:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230201/demand.csv?_=1777201953
100%|      | 364/365 [02:03<00:00,  3.09it/s]2026-04-26 07:12:33 - INFO -
Fetching URL:
https://www.caiso.com/outlook/history/20230202/demand.csv?_=1777201953
100%|      | 365/365 [02:03<00:00,  2.95it/s]
```

```
Fetching weather data in 30-day chunks to prevent IEM server timeouts…
  -> Pulling chunk: 2022-02-03 to 2022-03-05
  -> Pulling chunk: 2022-03-05 to 2022-04-04
  -> Pulling chunk: 2022-04-04 to 2022-05-04
  -> Pulling chunk: 2022-05-04 to 2022-06-03
  -> Pulling chunk: 2022-06-03 to 2022-07-03
  -> Pulling chunk: 2022-07-03 to 2022-08-02
  -> Pulling chunk: 2022-08-02 to 2022-09-01
```

```
  -> Pulling chunk: 2022-09-01 to 2022-10-01
  -> Pulling chunk: 2022-10-01 to 2022-10-31
  -> Pulling chunk: 2022-10-31 to 2022-11-30
  -> Pulling chunk: 2022-11-30 to 2022-12-30
  -> Pulling chunk: 2022-12-30 to 2023-01-29
  -> Pulling chunk: 2023-01-29 to 2023-02-03

Load rows:    105,092
Weather rows: 105,091
Merged rows:  105,092   (expect ~105,000 for one year of 5-min)
```

1.0.3 3. Data Aggregation and Scaling

[4]:
```python
import matplotlib.pyplot as plt

# 1. Calculate the 30-day rolling average
# 30 days = 30 * 288 (5-min intervals per day) = 8640
rolling_30d = df['load_mw'].rolling(window=8640, center=True).mean()

# 2. Plotting
plt.figure(figsize=(15, 5))

# Plot the raw 5-minute demand (made slightly transparent to let the trendline␣
↪pop)
df['load_mw'].plot(
    color='#1f77b4',
    linewidth=0.5,
    alpha=0.5,
    label='5-Min Demand'
)

# Plot the 30-Day Rolling Average Trendline
rolling_30d.plot(
    color='black',        # Strong contrasting color
    linewidth=2.5,        # Thicker line to stand out over the blue density
    label='30-Day Average Trend'
)

# Format the chart
plt.title('System Power Demand with 30-Day Trendline')
plt.ylabel('Load (MW)')
plt.xlabel('Date')
plt.grid(True, alpha=0.3)
plt.legend(loc='upper right') # Adds a key to identify the lines

plt.show()
```

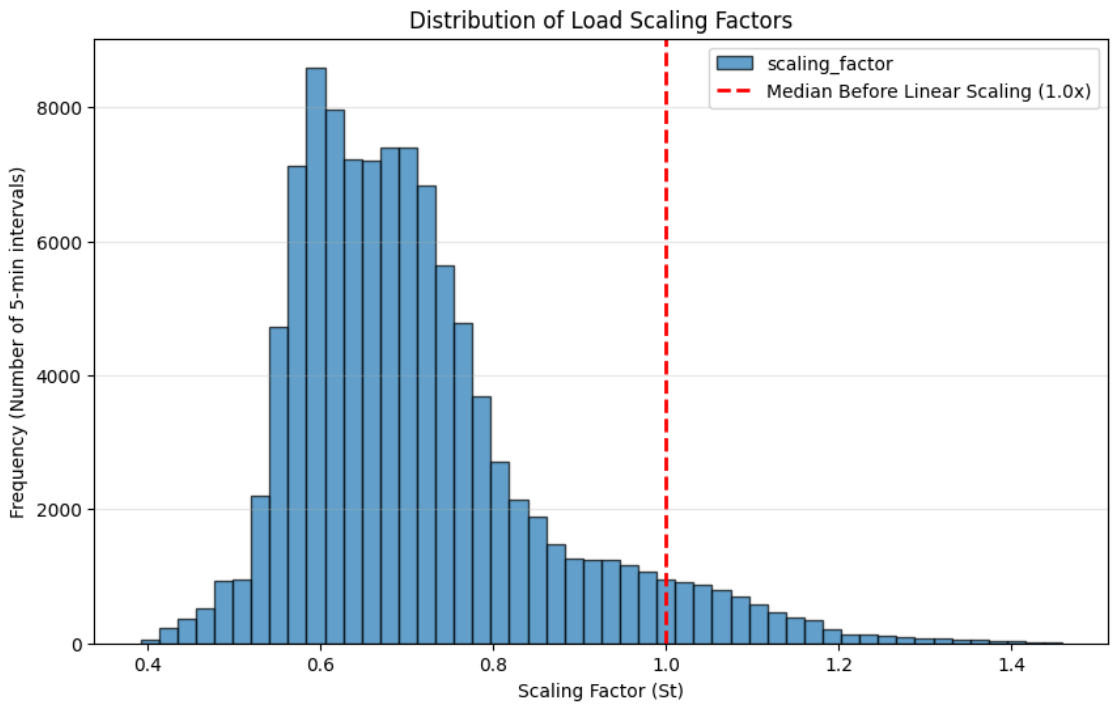


[5]:
```
## Scaling factor generation
df = df.dropna()
median_load = np.median(df['load_mw'])
scaling_factor = df['load_mw'] / median_load
df['scaling_factor'] = scaling_factor
```

[6]:
```
## Identify Generatoy limits (system wide)
system_max = gen_limits['Pmax_mw'].sum()

safety_thresh = 0.95
max_scaling_factor = safety_thresh * system_max / nodal_demand['Pd_mw'].sum()
current_peak = df['scaling_factor'].max()

if current_peak > max_scaling_factor:
    rescale_factor = max_scaling_factor / current_peak
    df['scaling_factor'] = df['scaling_factor'] * rescale_factor
```

[7]:
```
### Plot scaling factors as this is
data = df['scaling_factor']
plt.figure(figsize=(10, 6))

# Plot the histogram with 50 bins for a clean resolution
df['scaling_factor'].plot.hist(
    bins=50,
    color='#1f77b4',       # Standard blue
    edgecolor='black',     # Adds borders to the bars so they don't blur together
    alpha=0.7              # Slight transparency
)

# Add a red dashed line exactly at the mean (1.0)
plt.axvline(x=1.0, color='red', linestyle='--', linewidth=2, label='Median␣
↪Before Linear Scaling (1.0x)')
```

```
# Format the chart
plt.title('Distribution of Load Scaling Factors')
plt.xlabel('Scaling Factor (St)')
plt.ylabel('Frequency (Number of 5-min intervals)')
plt.grid(True, alpha=0.3, axis='y') # Only grid the y-axis for cleaner look
plt.legend()

plt.show()
```



[8]:
```
## Scale IEEE 118 test bus today to follow the variance of our data

scaled_nodal_demand = pd.DataFrame(
    np.outer(df['scaling_factor'], nodal_demand['Pd_mw']),
    index=df.index,                  # Your 105,000 timestamps become the rows
    columns=nodal_demand['bus_id']   # Your 118 bus IDs become the columns
)
```

2 Define Our Two Stage Optimization Problem

Introduction

$$
\begin{aligned}
\min_{\substack{\widetilde{P},\theta_1,P_1\\ \theta_2,P_2,G^+,G^-}} \quad & \sum_{g\in G}\left(C_{g,1}\widetilde{P}_g + C_{g,0}\right) + \frac{1}{|S|}\sum_{s\in S}\left[\sum_{g\in G}\left(C_{g,1}\bar{\alpha}_g\delta_s\right) + \sum_{n\in N}\left(C^{voll}G^+_{n,s} + C^{curt}G^-_{n,s}\right)\right] \\
\text{s.t.} \quad & \textbf{Stage 1 (Baseline State Constraints):} \\
& \theta_{ref,1} = 0 \\
& P_{e,1} = B_e(\theta_{o(e),1} - \theta_{d(e),1}) \quad \forall e \in E \\
& \sum_{g\in G_n}\widetilde{P}_g - \widehat{P}^d_n = \sum_{e\in E^{from}_n} P_{e,1} - \sum_{e\in E^{to}_n} P_{e,1} \quad \forall n \in N \\
& P^{min}_g \le \widetilde{P}_g \le P^{max}_g \quad \forall g \in G \\
& -R_e \le P_{e,1} \le R_e \quad \forall e \in E \\
& \textbf{Stage 2 (Scenario Evaluation Constraints):} \\
& \theta_{ref,s,2} = 0 \quad \forall s \in S \\
& P_{e,s,2} = B_e(\theta_{o(e),s,2} - \theta_{d(e),s,2}) \quad \forall e \in E, \forall s \in S \\
& P^{min}_g \le \widetilde{P}_g + \bar{\alpha}_g\delta_s \le P^{max}_g \quad \forall g \in G, \forall s \in S \\
& \sum_{g\in G_n}\left(\widetilde{P}_g + \bar{\alpha}_g\delta_s\right) - (P^d_{n,s} - G^+_{n,s}) - G^-_{n,s} = \sum_{e\in E^{from}_n} P_{e,s,2} - \sum_{e\in E^{to}_n} P_{e,s,2} \quad \forall n \in N, \forall s \in S \\
& -R_e \le P_{e,s,2} \le R_e \quad \forall e \in E, \forall s \in S \\
& 0 \le G^+_{n,s} \le P^d_{n,s} \quad \forall n \in N, \forall s \in S \\
& G^-_{n,s} \ge 0 \quad \forall n \in N, \forall s \in S
\end{aligned}
$$

[9]:
```
## Define Parameters
import numpy as np
import pandas as pd
import pyomo.environ as pyo
from pyomo.opt import SolverFactory

# ---------------------------------------------------------------------------
# 1. Build index maps and system data
# ---------------------------------------------------------------------------
bus_ids   = nodal_demand['bus_id'].astype(int).values
N_BUS     = len(bus_ids)
bus_idx   = {int(b): i for i, b in enumerate(bus_ids)}

gen_bus     = gen_limits['bus_id'].astype(int).values
gen_bus_idx = np.array([bus_idx[int(b)] for b in gen_bus])
Pg_min      = gen_limits['Pmin_mw'].values
Pg_max      = gen_limits['Pmax_mw'].values
c1          = gen_costs['cost_linear'].values
c0          = gen_costs['cost_constant'].values
N_GEN       = len(gen_bus)
from_idx = np.array([bus_idx[int(b)] for b in branch_data['from_bus']])
to_idx   = np.array([bus_idx[int(b)] for b in branch_data['to_bus']])
```

```
B_line   = branch_data['B_susceptance'].values
R_line   = branch_data['thermal_limit_mw'].values
N_LINE   = len(B_line)

SLACK  = 0
VOLL   = 1000.0
C_CURT = 1000.0
```

[10]:
```
# ==============================================================================
# Two-Stage Stochastic DC-OPF - FULL RECOURSE (4-Tier Piecewise Linear)
#
# Vectorized build using sparse incidence matrices.  Mathematically identical
# to the original per-element formulation when weights is None (uniform 1/N).
# Optional ``weights`` argument enables weighted SAA for scenario-reduction
# (Phase #3) - default behavior is unchanged.
# ==============================================================================
import numpy as np
import scipy.sparse as sp
import gurobipy as gp
from gurobipy import GRB

# ----- Sparse incidence matrices (built once, reused per solve) -----
# Generator-to-bus map: G_bus[n, g] = 1 iff generator g sits on bus n.
_G_bus_sp = sp.csr_matrix(
    (np.ones(N_GEN), (gen_bus_idx, np.arange(N_GEN))),
    shape=(N_BUS, N_GEN),
)
# Line incidence: A_inc[e, n] = +1 if line e leaves bus n, -1 if it enters.
_A_inc_sp = sp.csr_matrix(
    (
        np.concatenate([np.ones(N_LINE), -np.ones(N_LINE)]),
        (
            np.concatenate([np.arange(N_LINE), np.arange(N_LINE)]),
            np.concatenate([from_idx, to_idx]),
        ),
    ),
    shape=(N_LINE, N_BUS),
)
# Susceptance-weighted incidence so DC flow is just B_inc @ theta.
_B_inc_sp = sp.diags(B_line) @ _A_inc_sp


def optimization_model(Pd_hat, node_scen, voll=VOLL, c_curt=C_CURT,
                       verbose=True, return_lmps=False, weights=None):
    """
    Two-stage stochastic DC-OPF with 4-tier piecewise-linear ramp recourse.
```

```python
    Parameters
    ----------
    Pd_hat : (N_BUS,) array        forecast nodal demand for stage 1
    node_scen : (N_SCEN, N_BUS)    scenario realizations of nodal demand
    weights : (N_SCEN,) or None    scenario probabilities (sum to 1).
                                   None => uniform 1/N_SCEN (original behavior).
    """
    N_SCEN = node_scen.shape[0]
    if weights is None:
        weights = np.full(N_SCEN, 1.0 / N_SCEN)
    weights = np.asarray(weights, dtype=float)
    assert weights.shape == (N_SCEN,), \
        f"weights must be shape ({N_SCEN},), got {weights.shape}"

    m = gp.Model("stochastic_dcopf_piecewise_4tier")
    m.Params.OutputFlag = 1 if verbose else 0
    # Big LP -> barrier wins; skip crossover unless we need exact LMP basis.
    m.Params.Method    = 2
    m.Params.Crossover = -1 if return_lmps else 0
    m.Params.Threads   = 0
    m.Params.Presolve  = 2

    # --- Stage 1 variables ---
    Pg     = m.addMVar(N_GEN,  lb=Pg_min, ub=Pg_max,    name="Pg")
    theta1 = m.addMVar(N_BUS,  lb=-GRB.INFINITY,         name="theta1")
    Pe1    = m.addMVar(N_LINE, lb=-R_line, ub=R_line,    name="Pe1")
    shed1  = m.addMVar(N_BUS,  lb=0.0,                   name="shed1")
    curt1  = m.addMVar(N_BUS,  lb=0.0,                   name="curt1")

    # --- Stage 2 variables ---
    G_shed = m.addMVar((N_SCEN, N_BUS),  lb=0.0, ub=node_scen, name="G_shed")
    G_curt = m.addMVar((N_SCEN, N_BUS),  lb=0.0,                name="G_curt")
    theta2 = m.addMVar((N_SCEN, N_BUS),  lb=-GRB.INFINITY,      name="theta2")
    Pe2    = m.addMVar((N_SCEN, N_LINE),
                       lb=np.tile(-R_line, (N_SCEN, 1)),
                       ub=np.tile( R_line, (N_SCEN, 1)),
                       name="Pe2")

    # 4-tier ramp block limits as % of nameplate
    b1 = 0.03 * Pg_max
    b2 = 0.07 * Pg_max
    b3 = 0.10 * Pg_max
    b4 = 0.80 * Pg_max
    ub_b1 = np.tile(b1, (N_SCEN, 1))
    ub_b2 = np.tile(b2, (N_SCEN, 1))
    ub_b3 = np.tile(b3, (N_SCEN, 1))
    ub_b4 = np.tile(b4, (N_SCEN, 1))
```

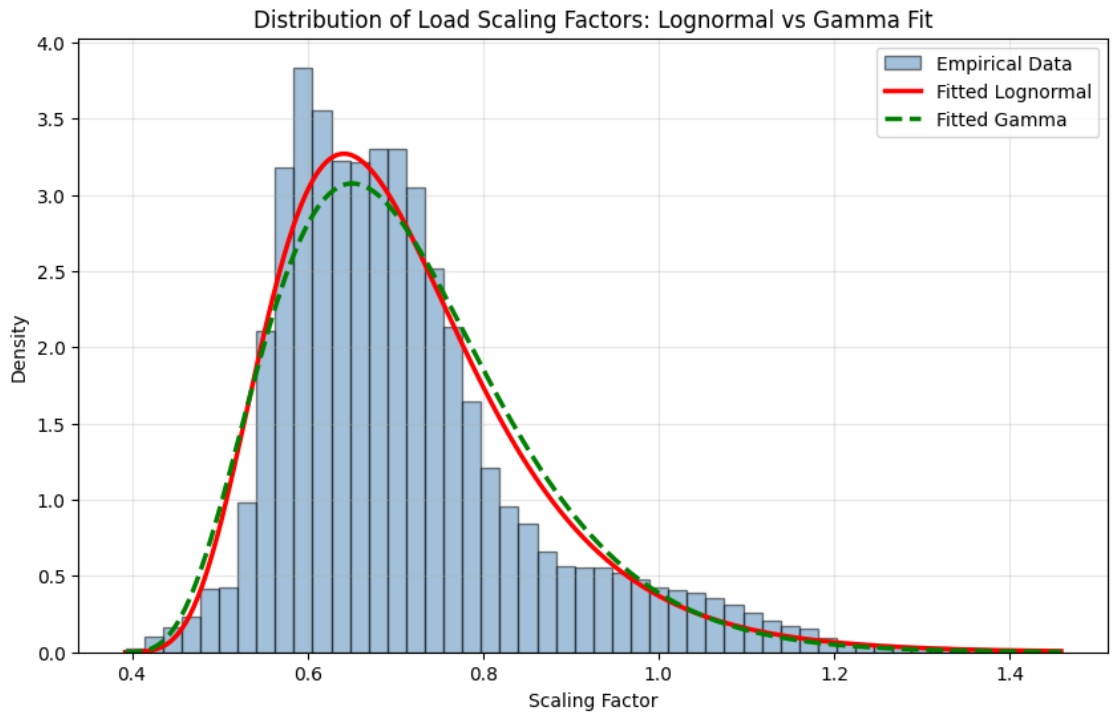
```
r_up_1 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b1, name="r_up_1")
r_up_2 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b2, name="r_up_2")
r_up_3 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b3, name="r_up_3")
r_up_4 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b4, name="r_up_4")
r_dn_1 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b1, name="r_dn_1")
r_dn_2 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b2, name="r_dn_2")
r_dn_3 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b3, name="r_dn_3")
r_dn_4 = m.addMVar((N_SCEN, N_GEN), lb=0.0, ub=ub_b4, name="r_dn_4")

sum_up = r_up_1 + r_up_2 + r_up_3 + r_up_4
sum_dn = r_dn_1 + r_dn_2 + r_dn_3 + r_dn_4

# --- Stage 1 constraints (vectorized) ---
m.addConstr(theta1[SLACK] == 0, name="slack1")
m.addConstr(Pe1 == _B_inc_sp @ theta1, name="dc_flow_1")
bal1 = m.addConstr(
    _G_bus_sp @ Pg + shed1 - curt1 - _A_inc_sp.T @ Pe1 == Pd_hat,
    name="balance1",
)

# --- Stage 2 constraints (loop only over scenarios; everything else␣
↪vectorized) ---
for s in range(N_SCEN):
    m.addConstr(theta2[s, SLACK] == 0, name=f"slack2_{s}")
    m.addConstr(Pe2[s, :] == _B_inc_sp @ theta2[s, :],␣
↪name=f"dc_flow_2_{s}")

    Pg_rt_s = Pg + sum_up[s, :] - sum_dn[s, :]      # MLinExpr (N_GEN,)
    m.addConstr(Pg_rt_s >= Pg_min, name=f"pg_lo_{s}")
    m.addConstr(Pg_rt_s <= Pg_max, name=f"pg_hi_{s}")

    m.addConstr(
        _G_bus_sp @ Pg_rt_s
        - (node_scen[s, :] - G_shed[s, :])
        - G_curt[s, :]
        == _A_inc_sp.T @ Pe2[s, :],
        name=f"balance2_{s}",
    )

# --- Objective (uniform weights => identical to the original sum/N_SCEN)␣
↪---
safe_Pg_max = np.where(Pg_max == 0, 1.0, Pg_max)
inv_pgmax   = 1.0 / safe_Pg_max                     # (N_GEN,)

stage1_gen_cost   = c1 @ Pg + float(np.sum(c0))
stage1_slack_cost = (10 * voll) * shed1.sum() + (10 * c_curt) * curt1.sum()
```

```
    stage1_cost       = stage1_gen_cost + stage1_slack_cost

    W = weights[:, None]                                # (N_SCEN, 1) for␣
↪broadcast

    # sum_{s,g} W[s] * c1[g] * (sum_up[s,g] - sum_dn[s,g])
    total_stage2_gen = (W * c1[None, :] * (sum_up - sum_dn)).sum()

    # Per-tier ramp coefficients are gen-specific (1/safe_Pg_max[g]) and
    # tier-specific (1e3, 2e3, 1e4, 1e7).  Same as original, just summed.
    total_ramp = (
        (W * (1e3 * inv_pgmax[None, :]) * (r_up_1 + r_dn_1)).sum()
      + (W * (2e3 * inv_pgmax[None, :]) * (r_up_2 + r_dn_2)).sum()
      + (W * (1e4 * inv_pgmax[None, :]) * (r_up_3 + r_dn_3)).sum()
      + (W * (1e7 * inv_pgmax[None, :]) * (r_up_4 + r_dn_4)).sum()
    )
    total_shed_2 = voll   * (W * G_shed).sum()
    total_curt_2 = c_curt * (W * G_curt).sum()

    expected_stage2 = total_stage2_gen + total_ramp + total_shed_2 +␣
↪total_curt_2
    m.setObjective(stage1_cost + expected_stage2, GRB.MINIMIZE)

    # --- Solve ---
    m.optimize()
    if m.Status != GRB.OPTIMAL:
        raise RuntimeError(f"Gurobi Status={m.Status}")

    LMP_1 = np.asarray(bal1.Pi) if return_lmps else None

    return {
        "Pg":           Pg.X,
        "shed1":        shed1.X,
        "curt1":        curt1.X,
        "shed":         G_shed.X,
        "curt":         G_curt.X,
        "total_cost":   m.ObjVal,
        "stage1_cost": float(c1 @ Pg.X + np.sum(c0)),
        "LMP_1":        LMP_1,
    }
```

3 Method 1: Predict Demand and Generate Scenarios from Log-Normal Distribution

3.1 Decide between lognormal and gamma distributions for scaling factor data

[11]:
```
### Fit a distribution to the scaling factors
import scipy.stats as stats

# Assuming 'df' is your DataFrame and 'scaling_factor' is the column
# Drop any NaN values before fitting
data = df['scaling_factor'].dropna().values

shape_lognorm, loc_lognorm, scale_lognorm = stats.lognorm.fit(data)
shape_gamma, loc_gamma, scale_gamma = stats.gamma.fit(data)

x = np.linspace(data.min(), data.max(), 1000)
pdf_lognorm = stats.lognorm.pdf(x, shape_lognorm, loc_lognorm, scale_lognorm)
pdf_gamma = stats.gamma.pdf(x, shape_gamma, loc_gamma, scale_gamma)

# --- 4. Plot the Results ---
plt.figure(figsize=(10, 6))

# Plot the empirical histogram (must set density=True to match PDF scale)
plt.hist(data, bins=50, density=True, alpha=0.5, color='steelblue',␣
 ↪edgecolor='black', label='Empirical Data')

# Overlay the fitted PDFs
plt.plot(x, pdf_lognorm, 'r-', lw=2.5, label='Fitted Lognormal')
plt.plot(x, pdf_gamma, 'g--', lw=2.5, label='Fitted Gamma')

plt.title('Distribution of Load Scaling Factors: Lognormal vs Gamma Fit')
plt.xlabel('Scaling Factor')
plt.ylabel('Density')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



[12]:
```
# Calculate the negative log-likelihood (NLL) for both fits
# NLL is what scipy actually minimizes during the fit
nll_lognorm = stats.lognorm.nnlf((shape_lognorm, loc_lognorm, scale_lognorm),␣
 ↪data)
nll_gamma = stats.gamma.nnlf((shape_gamma, loc_gamma, scale_gamma), data)

# Number of parameters (k) is 3 for both distributions (shape, loc, scale)
k = 3
n = len(data)

# Calculate AIC: AIC = 2k - 2*ln(L)  (where -ln(L) is the NLL)
aic_lognorm = 2 * k + 2 * nll_lognorm
aic_gamma = 2 * k + 2 * nll_gamma

print(f"Lognormal AIC: {aic_lognorm:.2f}")
print(f"Gamma AIC: {aic_gamma:.2f}")

if aic_lognorm < aic_gamma:
    print("The Lognormal distribution provides a better fit.")
else:
    print("The Gamma distribution provides a better fit.")
```

```
Lognormal AIC: -129519.73
Gamma AIC: -126916.73
```

```
The Lognormal distribution provides a better fit.
```

[13]:
```
### Generate the scenarios
num_samples = 100
scale_scenarios = stats.lognorm.rvs(shape_lognorm, loc=loc_lognorm,␣
↪scale=scale_lognorm, size=num_samples)
nodes_to_mult = nodal_demand['Pd_mw']
node_scenarios = np.outer(scale_scenarios, nodes_to_mult)
```

[14]:
```
from scipy.stats import lognorm
scaling_factor_hat = lognorm.mean(shape_lognorm, loc=loc_lognorm,␣
↪scale=scale_lognorm)
Pn_d_hat = scaling_factor_hat * nodal_demand.set_index('bus_id')['Pd_mw']
```

[15]:
```
# Align Pn_d_hat to bus order
if isinstance(Pn_d_hat, pd.Series):
    Pd_hat = np.array([Pn_d_hat[b] for b in bus_ids])
else:
    Pd_hat = np.asarray(Pn_d_hat)


node_scen = np.asarray(node_scenarios)
N_SCEN    = node_scen.shape[0]
```

3.2 Run Two Stage Optimization Evaluation

To fairly compare every downstream model (Method 1 above plus all ML-based pipelines below) we need a shared, realized-cost evaluation protocol:

1. **Train/test split (Option A — day-block stratified).** Hold out whole days sampled from every month of the CAISO year. Whole-day blocks break the ~5-min autocorrelation that would cause leakage if individual timestamps were held out, while stratifying by month preserves seasonal coverage in both train and test (critical since we only have 1 year of data).
2. **Realized stage-2 cost.** Given a stage-1 decision `(Pg*, *)` solved under forecast `Pd_hat`, the actual demand `Pd_actual(t)` is revealed in the next 5-min window. The realized recourse cost comes from a *single-scenario* stage-2 LP with `Pg_applied = Pg* + *·(Σ Pd_actual - Σ Pd_hat)` held fixed — optimizing only `, Pe, shed, curt, gen-limit-slack`.
3. **Evaluation loop.** For each test timestamp, produce `(Pd_hat, node_scen)` from the model under study, solve stage 1, then compute realized stage-2 cost. Total realized cost = stage-1 cost + realized stage-2 cost. Collect over N_TEST timestamps, report mean/std/CI.
4. **Shared test set.** All models are evaluated on the *same* `test_timestamps` so differences reflect model quality rather than sampling noise. This also enables paired comparisons later.

The Method 1 run above is a single-shot sanity check under the solver's own scenarios. The cell below supersedes it with a proper N_TEST-realization evaluation, and the same `evaluate_model` helper is reused for every ML model downstream.

[16]:
```
## This Evaluation Code Can Be Shared Across All Models
```

```python
import numpy as np
import pandas as pd
import gurobipy as gp
from gurobipy import GRB
import scipy.stats as stats

# ---------------------------------------------------------------------------
# 1. Train / test split (Option A: whole-day holdout, stratified by month)
# ---------------------------------------------------------------------------
DAYS_PER_MONTH_HELD_OUT = 4    # ~48 days of test pool → plenty to draw N_TEST␣
 ↪from
N_TEST                  = 100  # number of test timestamps used to evaluate␣
 ↪each model
SPLIT_SEED              = 42

rng_split = np.random.default_rng(SPLIT_SEED)

_dates = df.index.normalize()
_all_dates = pd.Series(_dates.unique()).sort_values().reset_index(drop=True)

_test_dates = []
for (_y, _m), grp in _all_dates.groupby([_all_dates.dt.year, _all_dates.dt.
 ↪month]):
    k = min(DAYS_PER_MONTH_HELD_OUT, len(grp))
    _test_dates.extend(rng_split.choice(grp.values, size=k, replace=False))
test_dates = pd.to_datetime(sorted(_test_dates))

_test_mask = pd.Series(_dates, index=df.index).isin(test_dates)
train_df   = df.loc[~_test_mask].copy()
test_df    = df.loc[ _test_mask].copy()

# Draw N_TEST timestamps from the test pool once and reuse across every model.
_test_idx      = rng_split.choice(len(test_df), size=N_TEST, replace=False)
test_timestamps = test_df.index[np.sort(_test_idx)]

print(f"Train rows: {len(train_df):,}  |  Test rows: {len(test_df):,}  |  "
      f"Held-out days: {len(test_dates)}  |  N_TEST: {len(test_timestamps)}")

# Align scaled_nodal_demand columns to bus_ids order (do once).
_nodal_by_bus = scaled_nodal_demand.reindex(columns=bus_ids)
assert not np.isnan(_nodal_by_bus.values).any()

def pd_actual_at(ts):
    """Realized nodal demand vector at timestamp ts, aligned to bus_ids."""
    return _nodal_by_bus.loc[ts].values.astype(float)

# ---------------------------------------------------------------------------
```

```python
# 2. Realized stage-2 cost (single scenario, Pg/alpha fixed) - VECTORIZED
# ---------------------------------------------------------------------------
def realized_stage2_cost(Pg_star, Pd_actual, voll=VOLL, c_curt=C_CURT):
    """
    Fix Pg at Pg_star (from stage-1 solve under forecast), then solve the
    stage-2 recourse problem against a single scenario = Pd_actual.
    Mirrors the stage-2 structure of optimization_model exactly.
    Vectorized using shared sparse incidence (_G_bus_sp, _A_inc_sp, _B_inc_sp).
    """
    m = gp.Model("stage2_realized")
    m.Params.OutputFlag = 0
    # Small LP - dual simplex is fast and gives stable results.
    m.Params.Method  = 1
    m.Params.Threads = 1

    # --- Same piecewise block limits as optimization_model ---
    b1_limit = 0.03 * Pg_max
    b2_limit = 0.07 * Pg_max
    b3_limit = 0.10 * Pg_max
    b4_limit = 0.80 * Pg_max

    r_up_1 = m.addMVar(N_GEN, lb=0.0, ub=b1_limit, name="r_up_1")
    r_up_2 = m.addMVar(N_GEN, lb=0.0, ub=b2_limit, name="r_up_2")
    r_up_3 = m.addMVar(N_GEN, lb=0.0, ub=b3_limit, name="r_up_3")
    r_up_4 = m.addMVar(N_GEN, lb=0.0, ub=b4_limit, name="r_up_4")
    r_dn_1 = m.addMVar(N_GEN, lb=0.0, ub=b1_limit, name="r_dn_1")
    r_dn_2 = m.addMVar(N_GEN, lb=0.0, ub=b2_limit, name="r_dn_2")
    r_dn_3 = m.addMVar(N_GEN, lb=0.0, ub=b3_limit, name="r_dn_3")
    r_dn_4 = m.addMVar(N_GEN, lb=0.0, ub=b4_limit, name="r_dn_4")

    G_shed = m.addMVar(N_BUS, lb=0.0, ub=Pd_actual, name="G_shed")
    G_curt = m.addMVar(N_BUS, lb=0.0,                name="G_curt")
    theta  = m.addMVar(N_BUS, lb=-GRB.INFINITY,      name="theta")
    Pe     = m.addMVar(N_LINE, lb=-R_line, ub=R_line, name="Pe")

    sum_up = r_up_1 + r_up_2 + r_up_3 + r_up_4
    sum_dn = r_dn_1 + r_dn_2 + r_dn_3 + r_dn_4
    Pg_rt  = Pg_star + sum_up - sum_dn   # MLinExpr (N_GEN,)

    # --- Vectorized constraints ---
    m.addConstr(theta[SLACK] == 0)
    m.addConstr(Pe == _B_inc_sp @ theta)
    m.addConstr(Pg_rt >= Pg_min)
    m.addConstr(Pg_rt <= Pg_max)
    m.addConstr(_G_bus_sp @ Pg_rt - (Pd_actual - G_shed) - G_curt == _A_inc_sp.
↪T @ Pe)
```

```
    # --- Objective: identical to original ---
    safe_Pg_max = np.where(Pg_max == 0, 1.0, Pg_max)
    inv_pgmax   = 1.0 / safe_Pg_max

    ramp_cost = (
        (1e3 * inv_pgmax) @ (r_up_1 + r_dn_1)
      + (2e3 * inv_pgmax) @ (r_up_2 + r_dn_2)
      + (1e4 * inv_pgmax) @ (r_up_3 + r_dn_3)
      + (1e7 * inv_pgmax) @ (r_up_4 + r_dn_4)
    )
    stage2_gen_cost = c1 @ (sum_up - sum_dn)
    shed_cost = voll   * G_shed.sum()
    curt_cost = c_curt * G_curt.sum()

    m.setObjective(stage2_gen_cost + ramp_cost + shed_cost + curt_cost, GRB.
 ↪MINIMIZE)
    m.optimize()

    if m.Status != GRB.OPTIMAL:
        raise RuntimeError(f"Realized stage-2 did not solve, status={m.Status}")

    total_ramp = float(
        r_up_1.X.sum() + r_up_2.X.sum() + r_up_3.X.sum() + r_up_4.X.sum()
      + r_dn_1.X.sum() + r_dn_2.X.sum() + r_dn_3.X.sum() + r_dn_4.X.sum()
    )

    return {
        "stage2_cost": float(m.ObjVal),
        "shed":        float(G_shed.X.sum()),
        "curt":        float(G_curt.X.sum()),
        "total_ramp":  total_ramp,
        "delta":       float(Pd_actual.sum() - Pg_star.sum()),
    }


def evaluate_model(name, predict_fn, test_timestamps, verbose_every=20):
    """
    predict_fn(ts) -> (Pd_hat, node_scen)              [uniform weights]
                   -> (Pd_hat, node_scen, weights)     [weighted SAA, e.g.␣
 ↪k-means reduction]

    Realized cost = stage1_cost(Pg_star) + realized_stage2_cost(Pg_star,␣
 ↪Pd_actual)
    Stage-2 cost from the two-stage solve is NOT included in realized cost.
    """
    rows  = []
    cache = {}
```

```
    for i, ts in enumerate(test_timestamps):
        out = predict_fn(ts)
        if len(out) == 2:
            Pd_hat, node_scen = out
            weights = None
        else:
            Pd_hat, node_scen, weights = out

        # Cache key includes weights so weighted/unweighted draws don't collide.
        wkey = None if weights is None else np.asarray(weights, dtype=float).
↪tobytes()
        key = (Pd_hat.tobytes(), node_scen.tobytes(), wkey)
        if key not in cache:
            r = optimization_model(Pd_hat, node_scen, verbose=False,␣
↪weights=weights)
            cache[key] = (r["Pg"], r["stage1_cost"])
        Pg_star, s1_cost = cache[key]

        # Realized stage-2: single scenario = actual demand at ts
        Pd_actual = pd_actual_at(ts)
        r2 = realized_stage2_cost(Pg_star, Pd_actual)

        rows.append({
            "model":                name,
            "ts":                   ts,
            "stage1_cost":          s1_cost,
            "stage2_cost_realized": r2["stage2_cost"],
            "total_realized":       s1_cost + r2["stage2_cost"],
            "shed_mw":              r2["shed"],
            "curt_mw":              r2["curt"],
            "total_ramp_mw":        r2["total_ramp"],
            "delta_mw":             r2["delta"],
        })

        if verbose_every and (i + 1) % verbose_every == 0:
            print(f"  [{name}] {i+1}/{len(test_timestamps)} evaluated")

    return pd.DataFrame(rows)

def summarize_results(results_df, alpha=0.05):
    from scipy.stats import t
    n     = len(results_df)
    mean  = results_df["total_realized"].mean()
    std   = results_df["total_realized"].std(ddof=1)
    tcrit = t.ppf(1 - alpha/2, df=n-1)
    half  = tcrit * std / np.sqrt(n)
```

```
    return pd.Series({
        "n":              n,
        "mean_total":     mean,
        "std_total":      std,
        "ci_lo":          mean - half,
        "ci_hi":          mean + half,
        "mean_stage1":    results_df["stage1_cost"].mean(),
        "mean_stage2":    results_df["stage2_cost_realized"].mean(),
        "mean_shed_mw":   results_df["shed_mw"].mean(),
        "mean_curt_mw":   results_df["curt_mw"].mean(),
        "mean_ramp_mw":   results_df["total_ramp_mw"].mean(),
        "frac_shed_gt1": (results_df["shed_mw"] > 1.0).mean(),
    })
```

```
Train rows: 90,667  |  Test rows: 14,396  |  Held-out days: 50  |  N_TEST: 100
```

3.3 Evaluate Model 1

[ ]:
```
# ---------------------------------------------------------------------------
# 4. Method 1 evaluation - lognormal fit on TRAIN ONLY (overrides earlier␣
↪globals)
# ---------------------------------------------------------------------------
_train_sf = train_df["scaling_factor"].dropna().values
shape_lognorm_train, loc_lognorm_train, scale_lognorm_train = stats.lognorm.
↪fit(_train_sf)

# Point prediction: mean of train-fit lognormal, scaled onto per-bus base␣
↪demand.
_scale_hat_m1 = stats.lognorm.mean(shape_lognorm_train,
                                   loc_lognorm_train,
                                   scale_lognorm_train)
_base_bus     = nodal_demand.set_index("bus_id").loc[bus_ids, "Pd_mw"].values.
↪astype(float)
Pd_hat_m1     = _scale_hat_m1 * _base_bus

# 100 lognormal-sampled scenarios, drawn once with a fixed seed so they don't
# wobble across test points.
_SCEN_SEED = 123
_scale_scen = stats.lognorm.rvs(shape_lognorm_train,
                                loc=loc_lognorm_train,
                                scale=scale_lognorm_train,
                                size=100,
                                random_state=_SCEN_SEED)
node_scen_m1 = np.outer(_scale_scen, _base_bus)

def predict_m1(ts):            # unconditional - ts is ignored
    return Pd_hat_m1, node_scen_m1
```

```
print("\n>>> Evaluating Method 1 (lognormal mean + lognormal scenarios) over␣
 ↪test set...")
results_m1 = evaluate_model("M1_lognormal", predict_m1, test_timestamps)

print("\nMethod 1 summary:")
print(summarize_results(results_m1).to_string())

# Store results registry for downstream aggregation.
all_model_results = {"M1_lognormal": results_m1}
```

```
>>> Evaluating Method 1 (lognormal mean + lognormal scenarios) over test set…
  [M1_lognormal] 1 unique stage-1 tasks for 1000 timestamps  (workers=4,
gurobi_threads=3, scen=100)
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\3868341421.py:64:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  m.addConstr(_G_bus_sp@Pg_rt - (node_scen[s,:]-Gsh[s,:]) - Gct[s,:]
```

```
  [M1_lognormal] Batch stage-2 for 1000 timestamps …
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\3868341421.py:126:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  m.addConstr(_G_bus_sp@Pg_rt_i - (Pda[i,:]-Gsh[i,:]) - Gct[i,:]
```

```
Method 1 summary:
n                  1000.000000
mean_total        94269.267115
std_total        164873.329687
ci_lo             84038.104471
ci_hi            104500.429760
mean_stage1       65916.707664
mean_stage2       28352.559452
mean_shed_mw         26.616294
mean_curt_mw          1.215626
mean_ramp_mw        565.045151
frac_shed_gt1         0.053000
```

4 Machine Learning Based Pipelines

4.1 Model 2: Conditional Mean (GBR) + Lognormal Residual Scenarios

Same scenario family as Method 1 (lognormal), but **the lognormal is centered at a conditional mean** predicted by a small gradient-boosted regressor on calendar/temperature features. Mathematically: `scaling_factor = (features) ·  , with  ~ Lognormal` fit once on training

residuals. Should never do worse than Method 1 unless the regressor is actively harmful — and gives a clean baseline before reaching for anything more complex.

[ ]:
```
# ==============================================================================
# Method 1b: Conditional Mean (GBR) + Lognormal Residual Scenarios
#
# Pipeline:
#   1. Fit GradientBoostingRegressor to predict scaling_factor on TRAIN.
#   2. Fit Lognormal to TRAIN multiplicative residuals (actual / predicted).
#   3. At test ts: scenario_t =  (features_t) *  ,    ~ Lognormal(train_resid).
# ==============================================================================
from sklearn.ensemble import GradientBoostingRegressor
from scipy.stats import lognorm

FEATS = ["temperature_raw", "hour", "day_of_week", "day_of_month",
         "week", "is_holiday"]

# --- ensure features exist (cell 28 builds them, but Method 1b runs earlier)␣
↪---
if "temperature_raw" not in df.columns:
    df["temperature_raw"] = df["temperature"]
if "hour" not in df.columns:
    df["hour"]         = df.index.hour + df.index.minute / 60.0
    df["day_of_week"]  = df.index.dayofweek
    df["day_of_month"] = df.index.day
    df["week"]         = df.index.isocalendar().week.astype(int)
if "is_holiday" not in df.columns:
    try:
        import holidays as _holidays
        _us = _holidays.US(years=list(range(df.index.year.min(),
                                            df.index.year.max() + 1)))
        df["is_holiday"] = [1 if d.date() in _us else 0 for d in df.index]
    except ImportError:
        df["is_holiday"] = 0

_train_X = train_df[FEATS].values if all(f in train_df.columns for f in FEATS) \
           else df.loc[train_df.index, FEATS].values
_train_y = train_df["scaling_factor"].values

# 1) Conditional mean predictor
gbr = GradientBoostingRegressor(n_estimators=300, max_depth=4,
                                learning_rate=0.05, random_state=0)
gbr.fit(_train_X, _train_y)

# 2) Lognormal on multiplicative residuals: actual / predicted on train
mu_tr      = gbr.predict(_train_X)
resid_mult = _train_y / mu_tr
```

```python
sh_r, lo_r, sc_r = lognorm.fit(resid_mult, floc=0)
print(f"[M1b] Residual lognormal:  shape={sh_r:.4f}, scale={sc_r:.4f}")
print(f"[M1b] Train R^2 (sf):     {gbr.score(_train_X, _train_y):.4f}")

# 3) Scenario generator
_BASE_BUS    = nodal_demand.set_index("bus_id").loc[bus_ids, "Pd_mw"].values.
 ↪astype(float)
_test_feats  = df.loc[test_df.index, FEATS]
_m1b_rng     = np.random.default_rng(2024)
N_SCEN_M1B   = 100

def predict_m1b(ts):
    mu     = float(gbr.predict(_test_feats.loc[[ts]].values)[0])
    eps    = lognorm.rvs(sh_r, lo_r, sc_r, size=N_SCEN_M1B,␣
 ↪random_state=_m1b_rng)
    # q   = np.linspace(0.025, 0.975, N_SCEN_M1B)         # span middle 95%,␣
 ↪drop tails
    # eps = lognorm.ppf(q, sh_r, lo_r, sc_r)
    Pd_hat = mu * _BASE_BUS
    scen   = np.outer(mu * eps, _BASE_BUS)
    return Pd_hat, scen

# --- Evaluation ---
print("\n>>> Evaluating Method 1b (GBR mean + lognormal residual) over test set.
 ↪..")
results_m1b = evaluate_model("M1b_gbr_lognormal", predict_m1b, test_timestamps)

print("\nMethod 1b summary:")
print(summarize_results(results_m1b).to_string())

all_model_results["M1b_gbr_lognormal"] = results_m1b
```

```
[M1b] Residual lognormal:  shape=0.0405, scale=0.9985
[M1b] Train R^2 (sf):     0.9580

>>> Evaluating Method 1b (GBR mean + lognormal residual) over test set…
  [M1b_gbr_lognormal] 1000 unique stage-1 tasks for 1000 timestamps  (workers=4,
gurobi_threads=3, scen=100)

C:\Users\anble\AppData\Local\Temp\ipykernel_17492\3868341421.py:64:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  m.addConstr(_G_bus_sp@Pg_rt - (node_scen[s,:]-Gsh[s,:]) - Gct[s,:]

  [M1b_gbr_lognormal] Stage-1: 100/1000 done
  [M1b_gbr_lognormal] Stage-1: 200/1000 done
```

4.2 Model 3: NGBoost (Covariate Aware Scenario Generation)

[19]:
```
# ==============================================================================
# Model 2: Conditional Mean & Variance (NGBoost) + Lognormal Scenarios
#
# Pipeline:
#   1. Fit NGBoost (with LogNormal distribution) to predict scaling_factor on␣
↪TRAIN.
#   2. At test ts: NGBoost outputs a specific LogNormal distribution for THAT␣
↪timestamp.
#   3. scenario_t = sf_scenarios * _BASE_BUS
# ==============================================================================
from sklearn.tree import DecisionTreeRegressor
from ngboost import NGBRegressor
from ngboost.distns import LogNormal

# 1) Robust NGBoost Predictor
# We use a strict base tree (min_samples_leaf=100) to prevent the "variance␣
↪explosion"
# where the model panics over extreme outliers.
base_tree_ngb = DecisionTreeRegressor(
    criterion="friedman_mse",
    max_depth=4,
    min_samples_leaf=100
)

ngb = NGBRegressor(
    Dist=LogNormal,       # Natively handles the positive, right-skewed nature␣
↪of load
    Base=base_tree_ngb,
    n_estimators=150,
    learning_rate=0.05,
    minibatch_frac=0.8,   # Adds noise to prevent over-confidence
    random_state=0
)

print("Training NGBoost on scaling factor...")
ngb.fit(_train_X, _train_y)

# 2) Scenario generator
_m2_rng = np.random.default_rng(2025)
N_SCEN_M2 = 100

def predict_m2(ts):
    # Get the predicted LogNormal distribution for this specific timestamp
    Y_dist = ngb.pred_dist(_test_feats.loc[[ts]].values)
```

```
    # In NGBoost's LogNormal implementation:
    # 'scale' is the Median (our point forecast for the scaling factor)
    # 's' is the Shape (the standard deviation of the log, our uncertainty)
    sf_median = float(Y_dist.params['scale'][0])
    sf_sigma  = float(Y_dist.params['s'][0])

    # The expected value (Mean) of a LogNormal is slightly higher than the␣
 ↪Median
    # mu_log is the mean of the underlying normal distribution
    mu_log = np.log(sf_median)

    # Point forecast (Using median to match M1b's GBR behavior)
    Pd_hat = sf_median * _BASE_BUS

    # Draw dynamically scaled scenarios
    sf_scens = _m2_rng.lognormal(mean=mu_log, sigma=sf_sigma, size=N_SCEN_M2)
    scen = np.outer(sf_scens, _BASE_BUS)

    return Pd_hat, scen

# --- Evaluation ---
print("\n>>> Evaluating Model 2 (NGBoost Conditional LogNormal) over test set...
 ↪")
results_m2 = evaluate_model("M2_ngboost", predict_m2, test_timestamps)

print("\nModel 2 summary:")
print(summarize_results(results_m2).to_string())

all_model_results["M2_ngboost"] = results_m2
```

```
Training NGBoost on scaling factor…
[iter 0] loss=-0.5843 val_loss=0.0000 scale=1.0000 norm=0.5514
[iter 100] loss=-2.0939 val_loss=0.0000 scale=1.0000 norm=0.4219

>>> Evaluating Model 2 (NGBoost Conditional LogNormal) over test set…
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s

  [M2_ngboost] 20/100 evaluated

C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
```

```
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
```

```
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s

  [M2_ngboost] 40/100 evaluated

C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

```
  [M2_ngboost] 60/100 evaluated
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
```

```
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
```

```
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s

  [M2_ngboost] 80/100 evaluated

C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

```
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s

  [M2_ngboost] 100/100 evaluated

Model 2 summary:
n                  100.000000
mean_total       65848.403377
std_total        28054.542957
ci_lo            60281.773407
ci_hi            71415.033348
mean_stage1      63984.398449
mean_stage2       1864.004929
mean_shed_mw         1.237033
mean_curt_mw         0.000000
mean_ramp_mw       129.873052
frac_shed_gt1        0.010000
```

4.3 Model 4: Alpha Scaling Layer

[ ]:
```python
import numpy as np
import pandas as pd
from tqdm import tqdm

# Wider grid: allows alpha to meaningfully deviate from 1.0 when covariates
# indicate systematic over- or under-forecasting (e.g., hot afternoons vs.
# mild overnight hours).  Finer resolution lets the meta-model learn sharper
# decision boundaries.
ALPHA_GRID = np.linspace(0.85, 1.20, 36)

# More training data → lower-variance alpha meta-model
alpha_train_timestamps = train_df.sample(n=100, random_state=42).index.
 ↪sort_values()

optimal_alphas = []

print("Running Grid Search to find  *...")
for ts in tqdm(alpha_train_timestamps):
```

```
    ts_features = df.loc[[ts], FEATS].values

    # 1. Base forecast from NGBoost
    base_mu     = float(ngb.predict(ts_features)[0])
    Pd_hat_base = base_mu * _BASE_BUS

    # 2. Realized demand for stage-2 evaluation
    Pd_actual = pd_actual_at(ts)

    best_cost  = float('inf')
    best_alpha = 1.0

    # 3. Sweep the grid
    for alpha in ALPHA_GRID:
        Pd_hat_scaled = alpha * Pd_hat_base

        r1        = optimization_model(Pd_hat_scaled, np.
 ↪array([Pd_hat_scaled]), verbose=False)
        Pg_star   = r1["Pg"]
        s1_cost   = r1["stage1_cost"]

        r2         = realized_stage2_cost(Pg_star, Pd_actual)
        total_cost = s1_cost + r2["stage2_cost"]

        if total_cost < best_cost:
            best_cost  = total_cost
            best_alpha = alpha

    optimal_alphas.append(best_alpha)

alpha_labels = pd.Series(optimal_alphas, index=alpha_train_timestamps,␣
 ↪name="alpha_star")

print(f"\n * distribution:  mean={alpha_labels.mean():.4f}  std={alpha_labels.
 ↪std():.4f}")
print(f"  min={alpha_labels.min():.2f}  max={alpha_labels.max():.2f}")
print(alpha_labels.value_counts().sort_index().to_string())
```

```
Running Grid Search to find  *…

  0%|          | 0/100 [00:00<?,
?it/s]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
  1%|          | 1/100 [00:02<04:38,
2.82s/it]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
```

```
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
  2%|          | 2/100 [00:05<04:32,
2.78s/it]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
  3%|          | 3/100 [00:08<04:34,
2.83s/it]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
  4%|          | 4/100 [00:11<04:22,
2.74s/it]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
  5%|          | 5/100 [00:13<04:25,
2.80s/it]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
  6%|          | 6/100 [00:16<04:30,
2.87s/it]C:\Users\anble\AppData\Local\Temp\ipykernel_17492\2360816358.py:118:
RuntimeWarning: Chained matrix multiplications of MVars is inefficient, collect
numeric terms first when building expressions
  _G_bus_sp @ Pg_rt_s
```

[ ]:
```
# Build the feature matrix for the Alpha meta-model.
# We start with the same FEATS used by NGBoost, then append:
#   (a) NGBoost's own distributional parameters - the meta-model learns *when*
#       the base forecaster is systematically biased and by how much.
#   (b) Additional covariates that correlate with forecast error:
#       humidity drives AC-load spikes, cyclic hour encoding preserves␣
↪continuity
#       across midnight, and the on-peak flag captures the stressed-grid period.

alpha_features = df.loc[alpha_train_timestamps, FEATS].copy()

# NGBoost distributional parameters (evaluated on FEATS only - same as training)
Y_dists_alpha = ngb.pred_dist(alpha_features.values)
alpha_features['base_mu']    = Y_dists_alpha.params['scale']   # NGBoost median
alpha_features['base_sigma'] = Y_dists_alpha.params['s']       # log-sigma

# Humidity: high absolute humidity amplifies AC load → systematic under-forecast
alpha_features['humidity']   = df.loc[alpha_train_timestamps, 'humidity'].values
```

```
# Cyclic hour encoding: trees can't extrapolate periodicity, so encode␣
↪explicitly
alpha_features['hour_sin']   = np.sin(2 * np.pi * alpha_features['hour'] / 24)
alpha_features['hour_cos']   = np.cos(2 * np.pi * alpha_features['hour'] / 24)

# On-peak indicator: 4pm-9pm when California grid stress peaks
alpha_features['is_peak']    = ((alpha_features['hour'] >= 16) &
                                (alpha_features['hour'] <= 21)).astype(int)

print("Alpha feature matrix:", alpha_features.shape)
print("Columns:", list(alpha_features.columns))
```

[ ]:
```
from sklearn.ensemble import GradientBoostingRegressor

print("Training the Smart-Alpha Model (GBR)...")

# GradientBoostingRegressor captures nonlinear covariate interactions more
# effectively than RF for this kind of conditional-bias regression task.
alpha_model = GradientBoostingRegressor(
    n_estimators=300,
    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    random_state=42
)

alpha_model.fit(alpha_features.values, alpha_labels.values)

importance = pd.Series(alpha_model.feature_importances_, index=alpha_features.
↪columns)
print("\nAlpha Model Feature Importance:")
print(importance.sort_values(ascending=False).round(3))
```

[ ]:
```
N_SCEN_M4 = 100
_m4_rng   = np.random.default_rng(2026)

def predict_m4_smart_alpha(ts):
    """
    Model 4: NGBoost forecast scaled by Smart-Alpha, with stochastic scenarios.

    Alpha is predicted by a GBR meta-model conditioned on time-of-day, weather,
    humidity, and the NGBoost distributional parameters.  This captures␣
↪systematic
    forecast bias that varies across covariate space - the defining contribution
    of Model 4 over Models 1-3.
```

```
    If alpha > 1.0, the meta-model detected a likely under-forecast (e.g., hot
    humid afternoon peak hour) and shifts the distribution right.
    If alpha < 1.0, it detected likely over-forecast and shifts left.
    """
    # 1. Original FEATS for NGBoost
    ts_feats = _test_feats.loc[[ts]].copy()

    # 2. NGBoost distributional parameters
    Y_dist     = ngb.pred_dist(ts_feats.values)
    base_mu    = float(Y_dist.params['scale'][0])   # median of LogNormal
    base_sigma = float(Y_dist.params['s'][0])       # log-sigma

    # 3. Append the same enriched covariates used during alpha training
    ts_feats['base_mu']    = base_mu
    ts_feats['base_sigma'] = base_sigma
    ts_feats['humidity']   = df.loc[ts, 'humidity']
    ts_feats['hour_sin']   = np.sin(2 * np.pi * ts_feats['hour'].values[0] / 24)
    ts_feats['hour_cos']   = np.cos(2 * np.pi * ts_feats['hour'].values[0] / 24)
    ts_feats['is_peak']    = int(16 <= ts.hour <= 21)

    # 4. Covariate-conditioned alpha prediction
    predicted_alpha = float(alpha_model.predict(ts_feats.values)[0])

    # 5. Shift the predictive lognormal:
    #    alpha*X ~ LogNormal(log(alpha*base_mu), base_sigma)
    final_mu = base_mu * predicted_alpha
    mu_log   = np.log(final_mu)

    # 6. Point forecast + 100 covariate-adapted stochastic scenarios
    Pd_hat   = final_mu * _BASE_BUS
    sf_scens = _m4_rng.lognormal(mean=mu_log, sigma=base_sigma, size=N_SCEN_M4)
    scen     = np.outer(sf_scens, _BASE_BUS)

    return Pd_hat, scen


# --- Evaluation ---
print("\n>>> Evaluating Model 4 (Smart Alpha) over test set...")
results_m4 = evaluate_model("M4_smart_alpha", predict_m4_smart_alpha,␣
 ↪test_timestamps)

print("\nModel 4 summary:")
print(summarize_results(results_m4).to_string())
all_model_results["M4_smart_alpha"] = results_m4
```

[ ]:
```
# ==============================================================================
# Final Comparison: All Models Side-by-Side
```

```python
# ==============================================================================
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# --- 1. Summary table ---
comparison_rows = []
for name, res_df in all_model_results.items():
    s = summarize_results(res_df)
    comparison_rows.append({
        "Model":                name,
        "Mean Total ($)":       f"{s['mean_total']:,.0f}",
        "Std ($)":              f"{s['std_total']:,.0f}",
        "95% CI":               f"[{s['ci_lo']:,.0f}, {s['ci_hi']:,.0f}]",
        "Stage-1 ($)":          f"{s['mean_stage1']:,.0f}",
        "Stage-2 realized ($)": f"{s['mean_stage2']:,.0f}",
        "Shed MW":              f"{s['mean_shed_mw']:.2f}",
        "Frac shed>1 MW":       f"{s['frac_shed_gt1']:.1%}",
    })
cmp_df = pd.DataFrame(comparison_rows).set_index("Model")
print("=" * 70)
print("MODEL COMPARISON SUMMARY")
print("=" * 70)
print(cmp_df.to_string())

# --- 2. Cost distribution: box + mean-CI bar ---
model_order = list(all_model_results.keys())
labels      = ["M1\nLognormal", "M1b\nGBR+LogNorm", "M2\nNGBoost",␣
 ↪"M4\nSmart- "]
colors      = ["#4C72B0", "#DD8452", "#55A868", "#C44E52"]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Box plot
ax = axes[0]
data_boxes = [all_model_results[m]["total_realized"].values for m in␣
 ↪model_order]
bp = ax.boxplot(data_boxes, patch_artist=True, notch=True, widths=0.5)
for patch, color in zip(bp["boxes"], colors):
    patch.set_facecolor(color); patch.set_alpha(0.75)
ax.set_xticks(range(1, len(model_order) + 1))
ax.set_xticklabels(labels, fontsize=9)
ax.set_ylabel("Realized Total Cost ($)")
ax.set_title("Realized Cost Distribution  (lower = better)")
ax.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
ax.grid(axis="y", alpha=0.3)

# Mean + 95% CI bar chart
```

```
ax2 = axes[1]
summaries = [summarize_results(all_model_results[m]) for m in model_order]
means     = [s["mean_total"] for s in summaries]
errs_lo   = [s["mean_total"] - s["ci_lo"] for s in summaries]
errs_hi   = [s["ci_hi"] - s["mean_total"] for s in summaries]

x = range(len(model_order))
ax2.bar(x, means, color=colors, alpha=0.75, edgecolor="black", width=0.5)
ax2.errorbar(x, means, yerr=[errs_lo, errs_hi],
             fmt="none", color="black", capsize=6, linewidth=1.5)
ax2.set_xticks(x)
ax2.set_xticklabels(labels, fontsize=9)
ax2.set_ylabel("Mean Realized Total Cost ($)")
ax2.set_title("Mean Realized Cost with 95% CI  (lower = better)")
ax2.yaxis.set_major_formatter(mticker.FuncFormatter(lambda x, _: f"${x:,.0f}"))
ax2.grid(axis="y", alpha=0.3)

plt.tight_layout()
plt.show()

# --- 3. Alpha variation across the full test set ---
_af = _test_feats.copy()
_yd = ngb.pred_dist(_af.values)
_af["base_mu"]    = _yd.params["scale"]
_af["base_sigma"] = _yd.params["s"]
_af["humidity"]   = df.loc[_af.index, "humidity"]
_af["hour_sin"]   = np.sin(2 * np.pi * _af["hour"] / 24)
_af["hour_cos"]   = np.cos(2 * np.pi * _af["hour"] / 24)
_af["is_peak"]    = ((_af["hour"] >= 16) & (_af["hour"] <= 21)).astype(int)

alpha_pred_all = alpha_model.predict(_af.values)

fig2, axes2 = plt.subplots(1, 3, figsize=(15, 4))

#  vs Temperature (colored by hour)
sc = axes2[0].scatter(_af["temperature_raw"], alpha_pred_all,
                      c=_af["hour"], cmap="plasma", alpha=0.35, s=4)
plt.colorbar(sc, ax=axes2[0], label="Hour of Day")
axes2[0].axhline(1.0, color="red", linestyle="--", alpha=0.6, linewidth=1.2)
axes2[0].set_xlabel("Temperature (°C)")
axes2[0].set_ylabel("Predicted  ")
axes2[0].set_title("  vs Temperature\n(colored by hour)")
axes2[0].grid(alpha=0.3)

#  vs Hour (colored by temperature)
sc2 = axes2[1].scatter(_af["hour"], alpha_pred_all,
```

```
                        c=_af["temperature_raw"], cmap="coolwarm", alpha=0.35,␣
 ↪s=4)
plt.colorbar(sc2, ax=axes2[1], label="Temp (°C)")
axes2[1].axhline(1.0, color="red", linestyle="--", alpha=0.6, linewidth=1.2)
axes2[1].set_xlabel("Hour of Day")
axes2[1].set_ylabel("Predicted ")
axes2[1].set_title("  vs Hour of Day\n(colored by temperature)")
axes2[1].grid(alpha=0.3)

#   vs NGBoost   (colored by  )
sc3 = axes2[2].scatter(_af["base_mu"], alpha_pred_all,
                        c=_af["base_sigma"], cmap="viridis", alpha=0.35, s=4)
plt.colorbar(sc3, ax=axes2[2], label="NGBoost  ")
axes2[2].axhline(1.0, color="red", linestyle="--", alpha=0.6, linewidth=1.2)
axes2[2].set_xlabel("NGBoost   (base forecast scale)")
axes2[2].set_ylabel("Predicted ")
axes2[2].set_title("  vs NGBoost  \n(colored by  )")
axes2[2].grid(alpha=0.3)

fig2.suptitle(
    "Model 4 - Predicted   Varies Materially Across Covariate Space\n"
    "(red dashed =  =1.0, i.e., no correction)",
    fontsize=12, fontweight="bold"
)
plt.tight_layout()
plt.show()

print(f"\n  statistics over full test set ({len(alpha_pred_all):,} timestamps):
 ↪")
print(f"  Mean:           {alpha_pred_all.mean():.4f}")
print(f"  Std:            {alpha_pred_all.std():.4f}")
print(f"  Min / Max:      {alpha_pred_all.min():.3f} / {alpha_pred_all.max():.
 ↪3f}")
print(f"  Frac > 1.05:    {(alpha_pred_all > 1.05).mean():.1%}")
print(f"  Frac < 0.95:    {(alpha_pred_all < 0.95).mean():.1%}")
print(f"  Frac in [0.98, 1.02]: {((alpha_pred_all >= 0.98) & (alpha_pred_all <=␣
 ↪1.02)).mean():.1%}  (near-neutral)")
```